Eシリーズストレージを使用したネットアップのBeeGFS

BeeGFS on NetApp with E-Series Storage

NetApp
August 01, 2022

This PDF was generated from https://docs.netapp.com/ja-jp/beegfs/index.html on August 01, 2022.
Always check docs.netapp.com for the latest.

目次

Eシリーズストレージを使用したネットアップのBeeGFS 1
はじめに 2
このサイトに含まれている情報 2
用語と概念 2
設計世代 3
解決策の概要 4
アーキテクチャの概要 5
技術要件 8
解決策 の設計を確認します 13
設計の概要 13
ハードウェア構成 13
ソフトウェア構成 14
設計の検証 18
サイジングガイドライン 21
パフォーマンスの調整 23
解決策 を導入します 26
導入の概要 26
Ansibleのインベントリを確認できます 27
ベストプラクティスを確認 29
ハードウェアを導入 32
ソフトウェアを導入 33
5つのビルディングブロックを超えた拡張性 70
ストレージプールのオーバープロビジョニングの割合を推奨します 71
法的通知 72
著作権 72
商標 72
特許 72
プライバシーポリシー 72
オープンソース 72

Eシリーズストレージを使用したネットアップのBeeGFS

はじめに

このサイトに含まれている情報

このサイトには、NetAppでBeeGFSを使用する第2世代のNetApp Verified Architecture（NVA）に関する情報が掲載されています。

このサイトの情報は次のとおりです。

- アーキテクチャの設計上の考慮事項は、アーキテクチャをサポートするために必要な機器、ケーブル、構成を決定する際に使用します。詳細については、を参照してください "設計の概要"。
- ブロックストレージシステム（ブロックノード）に接続された検証済みのBeeGFSファイルサーバ（ファイルノード）を使用した導入ガイドライン詳細については、を参照してください "導入の概要"。

このサイトは、解決策 の第1世代の設計および導入情報を含みません。前世代のコンテンツの詳細については、を参照してください "NVIDIA DGX A100 システムと BeeGFS を搭載した NetApp EF シリーズ AI"。

用語と概念

以下の用語と概念は、NetApp解決策 のBeeGFSに適用されます。

期間	説明
AI	人工知能
Ansible のインベントリ	目的のBeeGFS HAクラスタを記述するYAMLファイルを含むディレクトリ構造。
BMC の場合	ベースボード管理コントローラ：サービスプロセッサと呼ばれることもあります。
ブロックノード	ストレージシステム：
クライアント	サーバノード：
DL	ディープラーニング。
ファイルノード	BeeGFSファイルサーバ：
高可用性	高可用性：
HIC	ホストインターフェイスカード。
HPC	ハイパフォーマンスコンピューティング。

期間	説明
HPCスタイルのワークロード	HPCスタイルのワークロードの特徴は、通常、複数のコンピューティングノードまたはGPUがあり、すべてが同じデータセットに並行してアクセスする必要があることです。これにより、分散型のコンピューティングジョブやトレーニングジョブを円滑に進めることができます。多くの場合、このデータセットは大容量ファイルで構成されており、複数の物理ストレージノードにまたがってストライピングする必要があります。これにより、従来のハードウェアのボトルネックによって、単一ファイルへの同時アクセスが妨げられることがなくなります。
ml	機械学習。
NLP	自然言語処理。
NLU	自然言語の理解。
NVA	NetApp Verified Architecture（NVA）プログラムは、特定のワークロードとユースケースに対するリファレンス構成とサイジングに関するガイダンスを提供します。これらのソリューションは徹底的にテストされており、導入リスクを最小限に抑え、製品化サイクルを短縮するように設計されています。

設計世代

NetApp解決策 のBeeGFSは、現在第2世代の設計です。

第1世代と第2世代には、BeeGFSファイルシステムとNVMe EF600ストレージシステムを組み込んだベースアーキテクチャが含まれます。ただし、第2世代は、次のような追加の利点を最初に備えています。

- 2Uのラックスペースだけで、パフォーマンスと容量を2倍に向上
- 共有ディスクの2層ハードウェア設計に基づくハイアベイラビリティ（HA）
- NVIDIAのDGX A100 SuperPODの外部認定

第2世代のデザイン

ネットアップの第2世代BeeGFSは、ハイパフォーマンスコンピューティング（HPC）とHPCスタイルの機械学習（ML）、ディープラーニング（DL）、同様の人工知能（AI）手法など、要求の厳しいワークロードのパフォーマンス要件を満たすように最適化されています。共有ディスクのハイアベイラビリティ（HA）アーキテクチャを採用したBeeGFS on NetApp解決策 は、ワークロードやユースケースに合わせて拡張できるストレージを求めてダウンタイムやデータ損失を許容できない企業やその他の組織のデータ保持と可用性の要件にも対応します。この解決策 はネットアップによって検証されたばかりでなく、NVIDIA DGX SuperPODのストレージオプションとして外部認定にも合格しました。

最初の世代のデザイン

ネットアップの第1世代のBeeGFSは、NetApp EF600 NVMeストレージシステム、BeeGFS並列ファイルシステム、NVIDIA DGX™A100システム、NVIDIA®Mellanox®Quantum™QM8700 200Gbps IBスイッチを使用して、機械学習（ML）と人工知能（AI）のワークロード向けに設計されました。この設計では、ストレージとコンピューティングクラスタのインターコネクトファブリックに100Gbps InfiniBand（IB）を搭載しており、

ハイパフォーマンスワークロードに完全なIBベースのアーキテクチャを提供します。

第1世代の詳細については、を参照してください "NVIDIA DGX A100 システムと BeeGFS を搭載した NetApp EF シリーズ AI"。

解決策の概要

NetApp解決策 のBeeGFSは、BeeGFS並列ファイルシステムとNetApp EF600ストレージシステムを組み合わせることで、信頼性と拡張性に優れた対費用効果の高いインフラを実現し、要件の厳しいワークロードに対応します。

この設計では、最新のエンタープライズサーバおよびストレージハードウェアとネットワーク速度によって実現されるパフォーマンス密度を活用します。 デュアルAMD EPYC 7003「Milan」プロセッサを搭載し、200Gb（HDR）InfiniBandを使用して直接接続されたPCIe 4.0をサポートするファイルノードを必要とし、NVMe/IBプロトコルを使用してエンドツーエンドのNVMeおよびNVMeOFを提供するブロックノードを必要とします。

NVAプログラム

NetApp解決策 上のBeeGFSは、NetApp Verified Architecture（NVA）プログラムの一部であり、特定のワークロードとユースケースについて、参考構成とサイジングに関するガイダンスを提供します。NVAソリューションは、導入リスクを最小限に抑え、製品化サイクルを短縮するように徹底的にテストと設計されています。

ユースケース

以下のユースケースは、NetApp解決策 のBeeGFSに適用されます。

- 人工知能（AI）（機械学習（ML）、ディープラーニング（DL）、大規模な自然言語処理（NLP）、自然言語理解（NLU）など）詳細については、を参照してください "BeeGFS for AI：事実とフィクション"。
- MPI（メッセージ・パッシング・インターフェイス）やその他の分散コンピューティング技術により高速化されたアプリケーションを含む、ハイパフォーマンス・コンピューティング（HPC）。詳細については、を参照してください "BeeGFSがHPCの枠を超えている理由"。
- 次の特徴を持つアプリケーションワークロード：
  - 1GBを超えるファイルの読み取りまたは書き込み
  - 複数のクライアント（10s、100s、1000s）による同じファイルの読み取りと書き込み
- 数テラバイトまたは数ペタバイトのデータセット。
- 単一のストレージネームスペースが必要な環境：大容量ファイルと小容量ファイルを混在させる場合に最適化可能です。

利点

ネットアップでBeeGFSを使用する主なメリットは次のとおりです。

- 検証済みハードウェア設計の可用性：ハードウェアとソフトウェアコンポーネントを完全に統合し、予測可能なパフォーマンスと信頼性を確保します。
- Ansibleを使用して導入と管理を行い、シンプルさと大規模な一貫性を実現します。

- EシリーズPerformance AnalyzerおよびBeeGFSプラグインを使用した監視と監視が可能です。詳細については、を参照してください "NetApp Eシリーズソリューション監視フレームワークのご紹介"。
- データの保持と可用性を提供する共有ディスクアーキテクチャを採用した高可用性。
- コンテナとKubernetesを使用した最新のワークロード管理とオーケストレーションをサポート詳細については、を参照してください "Kubernetes BeeGFSを導入すれば、将来のニーズにも対応できる投資が実現します"。

HAアーキテクチャ

BeeGFS on NetAppは、共有ディスクハイアベイラビリティ（HA）アーキテクチャを実現するネットアップハードウェアと完全に統合された解決策 を作成することで、BeeGFSエンタープライズエディションの機能を拡張します。

> BeeGFSコミュニティエディションは無料でご利用いただけますが、エンタープライズエディションにはネットアップなどのパートナーからプロフェッショナルサポートサブスクリプション契約を購入する必要があります。エンタープライズエディションでは、耐障害性、クォータの適用、ストレージプールなど、いくつかの追加機能を使用できます。

次の図は、シェアードナッシングおよび共有ディスクHAアーキテクチャの比較です。

[]

詳細については、を参照してください "ネットアップがサポートするBeeGFSの高可用性についてお知らせします"。

Ansible

ネットアップのBeeGFSは、Ansible Automationを使用して提供および導入されます。この自動化はGitHubとAnsible Galaxy（BeeGFSコレクションはから入手できます "Ansible Galaxy" および "ネットアップのEシリーズGitHub"）。Ansibleは、主にBeeGFSビルディングブロックの構築に使用するハードウェアでテストされますが、サポートされているLinuxディストリビューションを使用して、ほぼすべてのx86ベースのサーバで実行するように設定できます。

詳細については、を参照してください "Eシリーズストレージを使用したBeeGFSの導入"。

アーキテクチャの概要

NetApp解決策 のBeeGFSには、検証済みのワークロードをサポートするために必要な機器、ケーブル配線、構成を決定するためのアーキテクチャ設計に関する考慮事項が含まれます。

ビルディングブロックアーキテクチャ

BeeGFSファイルシステムは、ストレージ要件に応じてさまざまな方法で導入および拡張できます。たとえば、主に多数の小さなファイルを扱うユースケースでは、メタデータのパフォーマンスと容量を強化できますが、大容量ファイルが少ないユースケースでは、実際のファイル内容よりも多くのストレージ容量とパフォーマンスを優先的に使用できます。このような複数の考慮事項は並列ファイルシステム環境のさまざまな次元に影響するため、ファイルシステムの設計と導入が複雑になります。

このような課題に対応するために、ネットアップでは、これらの要素のそれぞれをスケールアウトするための標準的なビルディングブロックアーキテクチャを設計しました。通常、BeeGFSビルディングブロックは、次の3つの設定プロファイルのいずれかに配置されます。

- BeeGFSの管理、メタデータ、ストレージサービスなど、単一のベースとなるビルディングブロックです
- BeeGFSメタデータとストレージビルディングブロック
- BeeGFSストレージのみのビルディングブロック

これらの3つのオプション間のハードウェア変更は、BeeGFSメタデータに小さいドライブを使用することだけです。それ以外の場合は、すべての設定変更がソフトウェアを介して適用されます。また、導入エンジンとしてAnsibleを使用することで、特定のビルディングブロックに必要なプロファイルを設定することで、構成タスクを簡単に実行できます。

詳細については、を参照してください [Verified hardware design]。

ファイルシステムサービス

BeeGFSファイルシステムには、次の主要サービスが含まれます。

- *管理サービス。*その他すべてのサービスを登録および監視します。
- *ストレージ・サービス。*データ・チャンク・ファイルと呼ばれる分散ユーザー・ファイルの内容を保存します。
- *メタデータサービス。*ファイルシステムのレイアウト、ディレクトリ、ファイル属性などを追跡します。
- *クライアント・サービス。*保存されたデータにアクセスするためのファイル・システムをマウントします。

次の図は、NetApp Eシリーズシステムで使用されるBeeGFS解決策 のコンポーネントと関係を示しています。

[]

BeeGFSは、並列ファイルシステムとして、複数のサーバノードを介してファイルをストライプ化することで、読み取り/書き込みのパフォーマンスと拡張性を最大化します。サーバノードは連携して動作するため、ほかのサーバノード（一般に_clients__）から同時にマウントしてアクセスすることができる単一のファイルシステムを提供します。これらのクライアントは、分散ファイルシステムをNTFS、XFS、ext4などのローカルファイルシステムと同様に認識して使用できます。

これら4つの主要サービスは、サポートされている幅広いLinuxディストリビューションで動作し、InfiniBand（IB）、Omni-Path（OPA）、RDMA over Converged Ethernet（RoCE）など、すべてのTCP/IPまたはRDMA対応ネットワークを介して通信します。BeeGFSサーバサービス（管理'ストレージ'メタデータ）はユーザ空間デーモンであり'クライアントはネイティブカーネルモジュール（パッチレス）ですすべてのコンポーネントは、リブートせずにインストールまたは更新でき、同じノード上で任意の組み合わせのサービスを実行できます。

ノードを確認しました

NetApp解決策 上のBeeGFSには、NetApp EF600ストレージシステム（ブロックノード）とLenovo ThinkSystem SR665サーバ（ファイルノード）という検証済みノードが含まれています。

ブロックノード：**EF600**ストレージシステム

NetApp EF600オールフラッシュアレイは、あらゆる数のワークロードを同時にサポートしながら、データへの一貫したほぼリアルタイムアクセスを提供します。EF600ストレージシステムは、データをAIアプリケーションやHPCアプリケーションに高速かつ継続的にフィードできるように、キャッシュされた最大200万IOPS、100マイクロ秒未満の応答時間、42GBpsのシーケンシャル読み取り帯域幅を1つのエンクロージャ内に提供します。

ファイルノード：**Lenovo ThinkSystem SR665 Server**

SR665は、PCIe 4.0を搭載した2ソケット2Uサーバです。この解決策 の要件を満たすように構成すると、直接接続されたEシリーズノードが提供するスループットとIOPSの可用性とバランスの取れた構成でBeeGFSファイルサービスを実行するための十分なパフォーマンスが得られます。

Lenovo SR665の詳細については、を参照してください "LenovoのWebサイト"。

ハードウェアの設計を確認した

このソリューションのビルディングブロック（次の図を参照）では、BeeGFSファイルレイヤに2台のデュアルソケットPCIe 4.0対応サーバを使用し、ブロックレイヤとして2台のEF600ストレージシステムを使用します。

[]

> 各ビルディングブロックには2つのBeeGFSファイルノードが含まれるため、フェイルオーバークラスタでクォーラムを確立するには少なくとも2つのビルディングブロックが必要です。2ノードクラスタを構成することもできますが、この構成にはフェイルオーバーの正常な実行を妨げる可能性がある制限があります。2ノードクラスタが必要な場合は、Tiebreakerとして第3のデバイスを組み込むことができます（ただし、この設計はこのサイトでは説明しません）。

各ビルディングブロックは、ファイルレイヤとブロックレイヤのフォールトドメインを分離する2層ハードウェア設計によって高可用性を実現します。各階層は個別にフェイルオーバーできるため、耐障害性が向上し、障害が連鎖的に発生するリスクを軽減できます。HDR InfiniBandとNVMeOFを併用することで、ファイルノードとブロックノードの間のスループットが向上し、レイテンシが最小限に抑えられます。また、完全な冗長性と十分なリンクのオーバーサブスクリプション機能を備えているため、システムが部分的に劣化している場合でも、分離型設計がボトルネックにならないよう

NetApp解決策 のBeeGFSは、導入環境のすべてのビルディングブロックで実行されます。最初に導入したビルディングブロックでは、BeeGFS管理、メタデータ、ストレージの各サービス（基本ビルディングブロックと呼ばれます）を実行する必要があります。以降のすべてのビルディングブロックは、BeeGFSメタデータとストレージサービスを実行するソフトウェアまたはストレージサービスのみを使用して構成されます。ビルディングブロックごとに異なる構成プロファイルを使用できるため、基盤となる同じハードウェアプラットフォームとビルディングブロック設計を使用して、ファイルシステムのメタデータやストレージ容量、パフォーマンスを拡張できます。

最大5つのビルディングブロックがスタンドアロンのLinux HAクラスタに統合され、クラスタリソースマネージャ（Pacemaker）ごとに適正な数のリソースを確保し、クラスタメンバーの同期を維持するために必要なメッセージングのオーバーヘッド（Corosync）を削減します。最低2つのビルディングブロックをクラスタごとに用意して、十分な数のメンバーがクォーラムを確立できるようにすることを推奨します。これらのスタンドアロンBeeGFS HAクラスタの1つ以上を組み合わせて、クライアントが単一のストレージネームスペースとしてアクセスできるBeeGFSファイルシステムを作成します（次の図を参照）。

[]

最終的にはラックあたりのビルディングブロック数は、特定のサイトの電力および冷却要件によって決まりますが、 解決策 は、1台の42Uラックに最大5個のビルディングブロックを導入できるように設計されていますが、ストレージ/データネットワークに使用する1U InfiniBandスイッチを2台収容するスペースがあります。各ビルディングブロックには8つのIBポート（冗長性のためにスイッチごとに4つ）が必要です。したがって、5つのビルディングブロックが40ポートHDR InfiniBandスイッチ（NVIDIA QM8700など）上の半分のポートを残して、脂肪ツリーまたは類似の非ブロッキングトポロジを実装できます。この構成により、ネットワークのボトルネックを発生させることなくストレージラックまたはコンピューティング/ GPUラックの数をスケールアップできます。オプションで、オーバーサブスクライブされたストレージファブリックをストレージファブリックベンダーの推奨事項で使用できます。

次の図は、80ノードのファットツリートポロジを示しています。

[]

Ansibleを導入エンジンとして使用してネットアップにBeeGFSを導入することで、管理者はコードの手法として最新のインフラを使用して環境全体を維持できます。これにより、複雑なシステムになる作業が大幅に簡易化されるため、管理者は設定をすべて一元的に定義して調整し、環境の規模に関係なく一貫して適用することができます。BeeGFSコレクションはから入手できます "Ansible Galaxy" および "ネットアップのEシリーズGitHub"。

技術要件

NetApp解決策 にBeeGFSを実装するには、使用する環境がテクノロジの要件を満たしていることを確認します。

ハードウェア要件

次の表に、NetApp解決策 上のBeeGFSに単一の第2世代ビルディングブロック設計を実装するために必要なハードウェアコンポーネントを示します。

> ⓘ 解決策 の特定の実装で使用されるハードウェアコンポーネントは、お客様の要件に応じて異なる場合があります。

カウント	ハードウェアコンポーネント	要件
2.	BeeGFSファイルノード	想定されるパフォーマンスを実現するには、各ファイルノードが次の構成を満たしているか、超えている必要があります。 プロセッサー： • AMD EPYC 7343 16C 3.2 GHz×2。 • 2つのNUMAゾーンとして構成されています。 メモリ： • 256GB。 • 16 x 16GB TruDDR4 3200MHz（2Rx8 1.2V）RDIMM A（より小さいDIMMよりも大きいDIMMを優先）。 • メモリ帯域幅を最大化するために実装。 • PCIe拡張: PCE Gen4 x16スロット×4：* • NUMAゾーンごとに2つのスロット。 • 各スロットには、Mellanox MCX653106A-HDATアダプタに十分な電力/冷却機能が備わっている必要があります。 その他: • OS用にRAID 1で構成された1TB 7.2K SATAドライブ（または同等のドライブ）2台。 • インバンドOS管理用の10GbE OCP 3.0アダプタ（または同等のアダプタ）。 • アウトオブバンドのサーバ管理用に、1GbE BMCとRedfish APIを使用。 • ホットスワップ対応のデュアル電源装置とパフォーマンスファン。 • ストレージInfiniBandスイッチに接続する必要がある場合は、Mellanox光InfiniBandケーブルをサポートしている必要があります。 • Lenovo SR665：* • カスタムのネットアップモデルには、デュアルポートMellanox ConnectX-6アダプタのサポートに必要なXClarityコントローラファームウェアのバージョンが含まれています。注文の詳細については、ネットアップにお問い合わせください。
8.	Mellanox ConnectX-6 HCA（ファイルノード用）	• MCX653106A-HDATホストチャネルアダプタ（HDR IB 200GB、デュアルポートQSFP56、PCIe4.0 x16）

カウント	ハードウェアコンポーネント	要件
8.	1M HDR InfiniBand ケーブル（ファイル/ブロックノードの直接接続用）。	• MCP1650 - H001E30（1 m Mellanoxパッシブカッパーケーブル、IB HDR、最大200Gbps、QSFP56、30AWG） 必要に応じて、ファイルノードとブロックノード間の距離を長くするために、長さを調整できます。
8.	HDR InfiniBandケーブル（ファイルノード/ストレージスイッチ接続用）	ファイルノードをストレージリーフスイッチに接続するために、適切な長さのInfiniBand HDRケーブル（QSFP56トランシーバ）が必要です。次のオプションがあります。 • MCP1650 - H002E26（2 m Mellanoxパッシブカッパーケーブル、IB HDR、最大200Gb/秒、QSFP56、30AWG）。 • MFS1S00-H003E（3 m Mellanoxアクティブファイバケーブル、IB HDR、最大200Gb/秒、QSFP56）。
2.	Eシリーズのブロックノード	次のように構成された2台のEF600コントローラ： • メモリ：256GB（コントローラあたり128GB）。 • アダプタ：2ポート200Gb / HDR（NVMe/IB） • ドライブ：必要な容量に合わせて構成

ソフトウェア要件

パフォーマンスと信頼性を予測するために、NetApp解決策 上のBeeGFSのリリースについて、解決策 の実装に必要な特定のバージョンのソフトウェアコンポーネントを使用してテストします。

ソフトウェア導入の要件

次の表に、AnsibleベースのBeeGFS導入の一環として自動的に導入されるソフトウェア要件を示します。

ソフトウェア	バージョン
BeeGFSの場合	7.2.6.
Corosync	3.1.5-1
ペースメーカー	2.1.0.8
OpenSMを使用します	opensm-5.9.0（MLNx_OFED 5.4-1.0.3.0から） ⓘ 仮想化を有効にするために直接接続する場合にのみ必要です。

Ansibleの制御ノード要件

NetApp解決策 のBeeGFSは、Ansible制御ノードから導入して管理します。詳細については、を参照してください "Ansibleのドキュメント"。

次の表に示すソフトウェア要件は、以下に記載するNetApp BeeGFSコレクションのバージョンに固有のものです。

ソフトウェア	バージョン
Ansible	2.11 pipを使用してインストールした場合：Ansible -4.7.0およびAnsible -core <2.12、>= 2.11.6
Python	3.9.
その他のPythonパッケージ	暗号化-350.0.0、netaddr-0.8.0
BeeGFS Ansibleコレクション	3.0.0

ファイルのノード要件

ソフトウェア	バージョン
Red Hat Enterprise Linuxの場合	Red Hat 8.4 Server Physical with High Availability（2ソケット）。 ファイルノードには、有効なRed Hat Enterprise Linux Serverサブスクリプションと Red Hat Enterprise Linux High Availabilityアドオンが必要です。
Linuxカーネル	4.18.0～305.25.1.el8_4.x86_64
InfiniBand / RDMAドライバ	受信トレイ
ConnectX-6 HCAファームウェア	FW：20.31.1014
PXE：3.6.0403	UEFI：14.24.0013

EF600ブロックノードの要件

ソフトウェア	バージョン
SANtricity OS の略	11.70.2
NVSRAM	N6000 - 872834 - D06.dlp
ドライブファームウェア	使用中のドライブモデルで最新バージョンが提供されています。

その他の要件

次の表に記載されている機器を検証に使用しましたが、必要に応じて適切な代替手段を使用できます。一般に、予期しない問題を回避するために、最新のソフトウェアバージョンを実行することを推奨します。

ハードウェアコンポーネント	ソフトウェアをインストールした
• Mellanox M8M8700 200Gb InfiniBandスイッチ×2	• ファームウェア3.9.2110
• Ansibleコントロールノード（仮想化）×1：* • プロセッサ：Intel（R）Xeon（R）Gold 6146 CPU @ 3.20 GHz • メモリ：8GB • ローカルストレージ：24GB	• CentOS Linux 8.4.2105 • カーネル4.18.0.305.3.1.el8.x86_64 インストールされているAnsibleとPythonのバージョンが上記の表のバージョンと同じです。
• 10x BeeGFSクライアント（CPUノード）：* • プロセッサ：3.0GHzでAMD EPYC 7302 16コアCPU x 1 • メモリ：128GB • ネットワーク：Mellanox MCX653106A-HDAT×2（アダプタごとに1ポート接続）	• Ubuntu 20.04 • カーネル:5.4.0-100-汎用 • InfiniBandドライバ：Mellanox OFED 5.4-1.0.3.0
• BeeGFSクライアント（GPUノード）×1：* • プロセッサ：2.25 GHzのAMD EPYC 7742 64コアCPU×2 • メモリ：1TB • ネットワーク：Mellanox MCX653106A-HDAT×2（アダプタごとに1ポート接続） このシステムはNVDIAS HGX A100プラットフォームをベースにしており、4つのA100 GPUを搭載しています。	• Ubuntu 20.04 • カーネル:5.4.0-100-汎用 • InfiniBandドライバ：Mellanox OFED 5.4-1.0.3.0

解決策 の設計を確認します

設計の概要

BeeGFS並列ファイルシステムとNetApp EF600ストレージシステムを組み合わせたNetApp解決策 上でBeeGFSをサポートするには、特定の機器、ケーブル配線、構成が必要です。

詳細はこちら。

- "ハードウェア構成"
- "ソフトウェア構成"
- "設計の検証"
- "サイジングガイドライン"
- "パフォーマンスの調整"

ハードウェア構成

ネットアップのBeeGFSのハードウェア構成には、ファイルノードとネットワークのケーブル配線が含まれます。

ファイルのノード構成

ファイルノードには、別々のNUMAゾーンとして構成された2つのCPUソケットがあり、同じ数のPCIeスロットとメモリへのローカルアクセスが含まれます。

InfiniBandアダプタは、適切なPCIライザーまたはスロットに装着する必要があります。これにより、使用可能なPCIeレーンとメモリチャネル間でワークロードが分散されます。個々のBeeGFSサービスの作業を特定のNUMAノードに完全に分離することで、ワークロードのバランスを調整します。目標は、各ファイルノードのパフォーマンスを、2つの独立したシングルソケットサーバと同様にすることです。

次の図は、ファイルノードのNUMA構成を示しています。

[]

BeeGFSプロセスは、使用するインターフェイスが同じゾーン内にあることを確認するために、特定のNUMAゾーンに固定されます。この構成により、ソケット間接続を介したリモートアクセスが不要になります。ソケット間接続はQPIまたはGMI2リンクと呼ばれることもあります。最新のプロセッサアーキテクチャであっても、HDR InfiniBandなどの高速ネットワークを使用する場合はボトルネックになる可能性があります。

ネットワークのケーブル構成

ビルディングブロック内では、各ファイルノードは合計4つの冗長InfiniBand接続を使用して2つのブロックノードに接続されます。また、各ファイルノードにInfiniBandストレージネットワークへの冗長接続が4つあります。

次の図に注意してください。

- 緑で示されているすべてのファイルノードポートは、ストレージファブリックへの接続に使用されます。他のすべてのファイルノードポートは、ブロックノードへの直接接続です。
- 特定のNUMAゾーン内の2つのInfiniBandポートは、同じブロックノードのAおよびBコントローラに接続します。
- NUMAノード0のポートは常に最初のブロックノードに接続します。
- NUMAノード1のポートは、2番目のブロックノードに接続します。

[]

> 冗長スイッチを使用するストレージネットワークでは、緑の薄いポートが1つのスイッチに接続され、緑のポートが別のスイッチに接続されます。

この図に示すケーブル構成では、BeeGFSサービスごとに次のことが可能です。

- BeeGFSサービスを実行しているファイルノードに関係なく、同じNUMAゾーンで実行します。
- 障害が発生した場所に関係なく、フロントエンドストレージネットワークおよびバックエンドブロックノードへのセカンダリの最適パスを確保する。
- ブロックノード内のファイルノードまたはコントローラのメンテナンスが必要な場合は、パフォーマンスへの影響を最小限に抑えます。

PCIeの完全な双方向帯域幅を利用するには、各InfiniBandアダプタの1つのポートをストレージファブリックに接続し、もう1つのポートをブロックノードに接続します。HDR InfiniBandポートの理論上の最大速度は25Gbpsです（シグナリングおよびその他のオーバーヘッドを考慮しない）。PCIe 4.0 x16スロットの最大シングル方向帯域幅は32Gbpsであり、理論的には50Gbpsの帯域幅を処理できるデュアルポートInfiniBandアダプタを搭載したファイルノードを実装する際にボトルネックとなる可能性があります。

次の図に、PCIeの双方向帯域幅をフルに活用するためのケーブル配線の設計を示します。

[]

BeeGFSサービスごとに、同じアダプタを使用して、クライアントトラフィックに使用する優先ポートと、そのサービスボリュームのプライマリ所有者であるブロックノードコントローラへのパスを接続します。詳細については、を参照してください "ソフトウェア構成"。

ソフトウェア構成

ネットアップのBeeGFSのソフトウェア設定には、BeeGFSネットワークコンポーネント、EF600ブロックノード、BeeGFSファイルノード、リソースグループ、BeeGFSサービスが含まれます。

BeeGFSネットワーク設定

BeeGFSネットワーク設定は、次のコンポーネントで構成されます。

- *フローティングIP *フローティングIPは、同じネットワーク内の任意のサーバーに動的にルーティングできる一種の仮想IPアドレスです。複数のサーバーが同じフローティングIPアドレスを所有できますが、一度にアクティブにできるのは1つのサーバーのみです。

各BeeGFSサーバサービスには、BeeGFSサーバサービスの実行場所に応じてファイルノード間を移動できる独自のIPアドレスがあります。このフローティングIP構成では、各サービスが他のファイルノードに独立してフェイルオーバーできます。クライアントは、特定のBeeGFSサービスのIPアドレスを知るだけで済み、そのサービスを現在実行しているファイルノードを認識する必要はありません。

- * BeeGFSサーバのマルチホーミング構成*解決策 の密度を高めるために'各ファイル・ノードには同じIPサブネットにIPが設定された複数のストレージ・インターフェイスがあります

  この設定がLinuxネットワークスタックで正常に機能するようにするには、追加の設定が必要です。これは、デフォルトでは、IPが同じサブネット内にある場合、1つのインターフェイスへの要求に別のインターフェイスで応答できるからです。他の欠点に加えて、このデフォルトの動作により、RDMA接続を適切に確立または維持することができなくなります。

  Ansibleベースの導入では、逆方向パス（RP）とアドレス解決プロトコル（ARP）の動作の締め付けに加え、フローティングIPの開始と停止のタイミングを保証します。対応するIPルートとルールが動的に作成され、マルチホームネットワークの設定が適切に機能できるようになります。

- * BeeGFSクライアントのマルチレール構成**Multi-rail*:複数の独立したネットワークレールを使用してパフォーマンスを向上させるアプリケーションの機能を指します

  BeeGFSはRDMAを接続に使用できますが、BeeGFSはIPoIBを使用してRDMA接続の検出と確立を簡素化します。BeeGFSクライアントが複数のInfiniBandインターフェイスを使用できるようにするには、異なるサブネット内にあるIPアドレスを各クライアントに設定し、各サブネット内のBeeGFSサーバサービスの半分に優先インターフェイスを設定します。

  次の図では、ライトグリーンで強調表示されているインターフェイスは1つのIPサブネット（たとえば、「100.127.0.0.0/16」）に配置され、ダークグリーンのインターフェイスは別のサブネット（たとえば「100.128.0.0/16」）に配置されています。

  次の図に、複数のBeeGFSクライアントインターフェイス間でのトラフィックの分散を示します。

  []

  BeeGFSの各ファイルは通常、複数のストレージサービスにまたがってストライピングされるため、マルチレール構成では、単一のInfiniBandポートでは実現できないスループットよりも高いスループットをクライアントに提供できます。たとえば、次のコード例は、クライアントが両方のインターフェイス間でトラフィックを分散できるようにする、一般的なファイルストライピング設定を示しています。

```
root@ictad21h01:/mnt/beegfs# beegfs-ctl --getentryinfo myfile
Entry type: file
EntryID: 11D-624759A9-65
Metadata node: meta_01_tgt_0101 [ID: 101]
Stripe pattern details:
+ Type: RAID0
+ Chunksize: 1M
+ Number of storage targets: desired: 4; actual: 4
+ Storage targets:
  + 101 @ stor_01_tgt_0101 [ID: 101]
  + 102 @ stor_01_tgt_0101 [ID: 101]
  + 201 @ stor_02_tgt_0201 [ID: 201]
  + 202 @ stor_02_tgt_0201 [ID: 201]
```

2つのIPoIBサブネットを使用することは、論理的に区別されます。必要に応じて、単一の物理InfiniBandサブネット（ストレージネットワーク）を使用できます。

> BeeGFS 7.3.0では、1つのIPoIBサブネットで複数のIBインターフェイスを使用できるように、マルチレールサポートが追加されました。NetApp解決策 のBeeGFSの設計は、BeeGFS 7.3.0が一般的に使用可能になる前に開発されたものであるため、2つのIPサブネットを使用してBeeGFSクライアントの2つのIBインターフェイスを使用する方法を示しています。マルチIPサブネットアプローチの利点の1つは、BeeGFSクライアントノードでマルチホーミングを設定する必要がないことです（詳細については、を参照してください） "BeeGFS RDMAのサポート"）。

EF600ブロックノード構成

ブロックノードは、同じドライブセットへの共有アクセスを持つ2つのアクティブ/アクティブRAIDコントローラで構成されます。通常、各コントローラにはシステムに設定されているボリュームの半分が所有されますが、必要に応じてもう一方のコントローラがテイクオーバーできます。

ファイルノード上のマルチパスソフトウェアは、各ボリュームへのアクティブなパスと最適パスを決定し、ケーブル、アダプタ、またはコントローラに障害が発生した場合に自動的に代替パスに移動します。

次の図は、EF600ブロックノードのコントローラのレイアウトを示しています。

[]

共有ディスクのHA解決策 を使用する場合、ボリュームが両方のファイルノードにマッピングされ、必要に応じて相互にテイクオーバーできるようになります。次の図は、パフォーマンスを最大化するためにBeeGFSサービスと優先ボリューム所有権を設定する例を示しています。各BeeGFSサービスの左側にあるインターフェイスは、クライアントとその他のサービスが接続に使用する優先インターフェイスを示します。

[]

前の例では、クライアントとサーバサービスは、インターフェイスi1bを使用してストレージサービス1と通信することを好みます。ストレージサービス1は、最初のブロックノードのコントローラA上のボリューム（storage_tgt_101、102）と通信するための優先パスとしてインターフェイスi1aを使用します。この構成では、InfiniBandアダプタで使用できる完全な双方向PCIe帯域幅を使用し、PCIe 4.0では実現できないデュアル

ポートHDR InfiniBandアダプタよりも優れたパフォーマンスを実現します。

BeeGFSファイルのノード設定

BeeGFSファイルノードは、複数のファイルノード間でBeeGFSサービスのフェイルオーバーを容易にするために、ハイアベイラビリティ（HA）クラスタに構成されます。

HAクラスタは、広く使用されている2つのLinux HAプロジェクトに基づいて設計されています。クラスタメンバーシップ用のCorosyncと、クラスタリソース管理用のPacemakerです。詳細については、を参照してください "高可用性アドオンに関するRed Hatトレーニング"。

複数のオープンクラスタフレームワーク（OCF）リソースエージェントがネットアップにオーサリングされ、拡張されました。このエージェントを使用すると、クラスタでBeeGFSリソースのインテリジェントな起動と監視が可能になります。

BeeGFS HA clusters（BeeGFS HAクラスタ

通常、BeeGFSサービスを（HAの有無にかかわらず）開始するときは、次のようなリソースが必要になります。

- サービスに到達できるIPアドレス。通常はNetwork Managerによって設定されます。
- BeeGFSがデータを格納するためのターゲットとして使用する基盤となるファイルシステム。

  これらは通常'/etc/fstabで定義され'システム・ディスクでマウントされます

- 他のリソースの準備ができたときにBeeGFSプロセスを開始するシステムdサービス。

  追加のソフトウェアがない場合、これらのリソースは単一のファイルノードからのみ開始されます。したがって、ファイルノードがオフラインになると、BeeGFSファイルシステムの一部にアクセスできなくなります。

複数のノードでBeeGFSサービスを起動できるため、ペースメーカーは各サービスと依存するリソースが一度に1つのノードでのみ動作していることを確認する必要があります。たとえば、2つのノードが同じBeeGFSサービスを起動しようとすると、どちらも基盤となるターゲット上の同じファイルに書き込みを試みると、データが破損するおそれがあります。この状況を回避するために、Pacemakerは、クラスタ全体の状態をすべてのノードで確実に同期し、クォーラムを確立するために、Corosyncに依存しています。

クラスタで障害が発生すると、Pacemakerは別のノードのBeeGFSリソースに反応して再起動します。一部の状況では、ペースメーカーが障害のある元のノードと通信できず、リソースが停止していることを確認できない場合があります。BeeGFSリソースを他の場所から再起動する前にノードが停止していることを確認するために、Pacemakerは障害のあるノードをフェンシングします。この場合、電源を切断します。

多くのオープンソースフェンシングエージェントを使用すると、Pacemakerは配電ユニット（PDU）を搭載したノードを遮断したり、RedfishなどのAPIを搭載したサーバベースボード管理コントローラ（BMC）を使用してノードを遮断したりできます。

HAクラスタでBeeGFSを実行している場合は、すべてのBeeGFSサービスと基盤となるリソースがペースメーカーによってリソースグループで管理されます。各BeeGFSサービスとそれが依存するリソースは、リソースグループに設定されます。これにより、リソースが正しい順序で開始および停止され、同じノードに配置されるようになります。

BeeGFSリソースグループごとに、PacemakerはカスタムのBeeGFSモニタリングリソースを実行します。こ

のリソースは、障害状態を検出し、特定のノードでBeeGFSサービスがアクセスできなくなったときにフェイルオーバーをインテリジェントにトリガーします。

次の図に、Pacemaker制御のBeeGFSサービスと依存関係を示します。

[]

> 同じタイプの複数のBeeGFSサービスが同じノードで起動するように、Pacemakerはマルチモード設定方式を使用してBeeGFSサービスを開始するように設定されます。詳細については、を参照してください "マルチモードでのBeeGFSのマニュアル"。

BeeGFSサービスは複数のノードで起動できる必要があるため'各サービスの構成ファイル（通常は/etc/beegfsにあります）は'そのサービスのBeeGFSターゲットとして使用されるEシリーズボリュームの1つに保存されますこれにより、特定のBeeGFSサービスのデータとともに、サービスの実行に必要なすべてのノードから設定へのアクセスが可能になります。

```
# tree stor_01_tgt_0101/ -L 2
stor_01_tgt_0101/
├── data
│   ├── benchmark
│   ├── buddymir
│   ├── chunks
│   ├── format.conf
│   ├── lock.pid
│   ├── nodeID
│   ├── nodeNumID
│   ├── originalNodeID
│   ├── targetID
│   └── targetNumID
└── storage_config
    ├── beegfs-storage.conf
    ├── connInterfacesFile.conf
    └── connNetFilterFile.conf
```

設計の検証

NetApp解決策 のBeeGFSの第2世代設計は、3つのビルディングブロック構成プロファイルを使用して検証されました。

構成プロファイルには、次のものが含まれます。

- BeeGFSの管理、メタデータ、ストレージサービスなど、単一のベースとなるビルディングブロックです。
- BeeGFSメタデータとストレージビルディングブロック
- BeeGFSストレージ専用のビルディングブロック。

このビルディングブロックは、2台のMellanox Quantum InfiniBand（M8700）スイッチに接続されました。また、10個のBeeGFSクライアントもInfiniBandスイッチに接続し、総合ベンチマークユーティリティの実行に使用しました。

次の図は、NetApp解決策 でBeeGFSを検証するために使用するBeeGFS設定を示しています。

[]

BeeGFSファイルのストライピング

並列ファイルシステムの利点は、複数のストレージターゲットに個別のファイルをストライプすることです。これは、同一または異なる基盤となるストレージシステム上のボリュームを表すことができます。

BeeGFSでは'各ファイルに使用するターゲットの数を制御し'各ファイルストライプに使用するチャンクサイズ（またはブロックサイズ）を制御するために'ディレクトリ単位およびファイル単位でストライピングを構成できますこの構成では、サービスを再設定したり再開したりすることなく、ファイルシステムがさまざまなタイプのワークロードやI/Oプロファイルをサポートできます。ストライプ設定を適用するには'beegfs-ctl'コマンドラインツールを使用するか'ストライピングAPIを使用するアプリケーションを使用します詳細については、のBeeGFSのマニュアルを参照してください "ストライピング" および "ストライピングAPI"。

最高のパフォーマンスを得るために、テスト全体を通してストライプパターンを調整し、各テストに使用するパラメータを記録しました。

IOR帯域幅テスト：複数のクライアント

IOR帯域幅テストでは、OpenMPIを使用して、統合I/OジェネレータツールIOR（から入手可能）の並列ジョブを実行しました "HPC GitHub"）を使用して、10のすべてのクライアントノードから1つ以上のBeeGFSビルディングブロックにアクセスします。特に明記されていない限り：

- すべてのテストで、直接I/Oを使用し、転送サイズは1MiBに設定しました。
- BeeGFSファイルのストライピングは'1 MBのチャンクサイズと1ファイルあたり1つのターゲットに設定されました

IORでは次のパラメータを使用し、1つのビルディングブロックではアグリゲートのファイルサイズが5TiB、3つのビルディングブロックでは40TiBにセグメント数を調整しました。

```
mpirun --allow-run-as-root --mca btl tcp -np 48 -map-by node -hostfile
10xnodes ior -b 1024k --posix.odirect -e -t 1024k -s 54613 -z -C -F -E -k
```

次の図に、1つのBeeGFSベース（管理、メタデータ、ストレージ）ビルディングブロックを使用したIORテスト結果を示します。

[]

次の図は、1つのBeeGFSメタデータとストレージビルディングブロックを使用したIORテスト結果を示しています。

[]

次の図に、1つのBeeGFSストレージ専用ビルディングブロックを使用したIORテスト結果を示します。

[]

次の図に、3つのBeeGFSビルディングブロックを使用したIORテスト結果を示します。

[]

基本となるビルディングブロックと後続のメタデータとストレージビルディングブロックのパフォーマンスの違いは、想定どおりごくわずかです。メタデータとストレージのビルディングブロックおよびストレージ専用のビルディングブロックを比較した場合、ストレージターゲットとして使用される追加のドライブが原因で読み取りパフォーマンスがわずかに向上します。ただし、書き込みパフォーマンスに大きな違いはありません。パフォーマンスを向上させるには、複数のビルディングブロックを1つに追加して、パフォーマンスを直線的に拡張します。

IOR帯域幅テスト：単一クライアント

IOR帯域幅テストでは、OpenMPIを使用して、1台の高性能GPUサーバを使用して複数のIORプロセスを実行し、1台のクライアントで実現可能なパフォーマンスを検証しました。

このテストでは'クライアントがLinuxカーネルのページキャッシュ(`tuneFileCacheType=native')を使用するように構成されている場合に'BeeGFSの再読み取り動作とパフォーマンスを'デフォルトのバッファ設定と比較します

ネイティブ・キャッシュ・モードでは'クライアント上のLinuxカーネル・ページ・キャッシュを使用するため'ネットワーク上で再送信されるのではなく'ローカル・メモリから再読み取り操作を行うことができます

次の図は、3つのBeeGFSビルディングブロックと1つのクライアントを使用したIORテスト結果を示しています。

[]

> これらのテストのBeeGFSストライピングは'ファイルあたり8つのターゲットを持つ1MBのチャンクサイズに設定されました

デフォルトのバッファモードを使用した場合の書き込みと初期の読み取りパフォーマンスは向上しますが、同じデータを複数回再読み取りするワークロードでは、ネイティブのキャッシュモードを使用するとパフォーマンスが大幅に向上します。ディープラーニングなどのワークロードで多くの期間にわたって同じデータセットを複数回再読み取りする場合は、読み取りパフォーマンスの向上が重要になります。

メタデータパフォーマンステスト

メタデータパフォーマンステストでは、MDTestツール（IORの一部として含まれる）を使用してBeeGFSのメタデータパフォーマンスを測定しました。テストでは、OpenMPIを使用して、10個のクライアントノードすべてで並列ジョブを実行しました。

以下のパラメータを使用して、ステップ2xでステップ10から320まで、ファイルサイズ4Kのプロセスの合計数を使用して、ベンチマークテストを実行しました。

```
mpirun -h 10xnodes –map-by node np $processes mdtest -e 4k -w 4k -i 3 -I
16 -z 3 -b 8 -u
```

メタデータパフォーマンスは、まずメタデータとストレージのビルディングブロックを2つずつ使用して測定

し、ビルディングブロックを追加してパフォーマンスがどのようにスケールアップするかを示しました。

次の図は、BeeGFSメタデータとストレージビルディングブロックを1つずつ使用したMDTestの結果を示しています。

[]

次の図は、2つのBeeGFSメタデータとストレージビルディングブロックを使用したMDTestの結果を示しています。

[]

機能検証

このアーキテクチャの検証の一環として、ネットアップは次の機能テストをいくつか実施しました。

- スイッチポートを無効にして、単一のクライアントInfiniBandポートを障害状態にします。
- スイッチポートを無効にして、単一サーバのInfiniBandポートを障害状態にします。
- BMCを使用した即時サーバ電源オフのトリガー
- ノードを正常にスタンバイにし、別のノードにサービスをフェイルオーバーします。
- ノードを正常にオンラインに戻し、元のノードにサービスをフェイルバックします。
- PDUを使用している一方のInfiniBandスイッチの電源をオフにします。すべてのテストは、BeeGFSクライアントで設定された「sysSessionChecksEnabled：false」パラメータを使用して、ストレステストの実行中に実行されました。エラーやI/Oの中断は発生しませんでした。

> 既知の問題 がある（を参照） "変更ログ") BeeGFSクライアント/サーバRDMA接続が予期せず中断される場合は、プライマリインターフェイスの喪失（「connInterfacesFile」で定義）またはBeeGFSサーバの障害のいずれかによって、アクティブなクライアントI/Oが最大10分間ハングアップしてから再開します。この問題 は、計画的メンテナンスのためにBeeGFSノードが正常に配置され、スタンバイ状態から外れたとき、またはTCPが使用中のときは発生しません。

NVIDIA DGX A100 SuperPODの検証

ネットアップでは、3つのビルディングブロックにメタデータとストレージ構成プロファイルが適用されたBeeGFSファイルシステムを使用して、NVDIA DGX A100 SuperPOD向けのストレージ解決策 の検証を実施しました。認定には、このNVAで説明した解決策 を、さまざまなストレージ、機械学習、ディープラーニングのベンチマークを実行している20台のDGX A100 GPUサーバでテストすることが含まれます。

詳細については、を参照してください "NVIDIA DGX SuperPODとネットアップ"。

サイジングガイドライン

BeeGFS解決策 には、検証テストに基づくパフォーマンスと容量のサイジングに関する推奨事項が含まれます。

ビルディングブロックアーキテクチャの目的は、特定のBeeGFSシステムの要件を満たす複数のビルディングブロックを追加することで、サイズを簡単に決定できる解決策 を作成することです。以下のガイドラインを使用して、環境の要件を満たすために必要なBeeGFSビルディングブロックの数とタイプを見積もります。

これらの見積もりは、ケースのパフォーマンスが最も優れていることに留意してください。統合ベンチマークアプリケーションは、基盤となるファイルシステムの使用を最適化するために作成され、実際のアプリケーションでは使用されない可能性があります。

パフォーマンスのサイジング

次の表に、推奨されるパフォーマンスサイジングを示します。

構成プロファイル	読み取り：**1MiB**	**1MiB**の書き込み
メタデータ+ストレージ	62GiBps	21GiBps
ストレージのみ	64GiBps	21GiBps

メタデータ容量のサイジングの推定値は、500GBの容量でBeeGFSに約1億5、000万ファイルを格納できる「経験則」に基づいています。（詳細については、のBeeGFSのマニュアルを参照してください "システム要件". ）

アクセス制御リストやディレクトリあたりのディレクトリ数やファイル数などの機能を使用すると、メタデータスペースの消費速度にも影響します。ストレージ容量の概算値は、使用可能なドライブ容量と、RAID 6およびXFSオーバーヘッドを考慮しています。

メタデータおよびストレージのビルディングブロックの容量サイジング

次の表に、メタデータ用の推奨容量およびストレージのビルディングブロックを示します。

ドライブサイズ（**2+2 RAID 1**）のメタデータボリュームグループ	メタデータ容量（ファイル数）	ドライブサイズ（**8+2 RAID 6**）ストレージボリュームグループ	ストレージ容量（ファイル内容）
1.92TB	1,797,120,000	1.92TB	47.91TB
3.84TB	3,594,240,000人	3.84TB	95.84TB
7.68TB	7、557、120、000	7.68TB	1、01.5 TB
15.3TB	16,156,800,000	15.3TB	430.84TB

メタデータとストレージビルディングブロックのサイジングを行う場合、メタデータボリュームグループとストレージボリュームグループに使用するドライブを小型化することでコストを削減できます。

ストレージ専用のビルディングブロックの容量サイジング

次の表に、ストレージ専用のビルディングブロックの容量をルールベースの設定値で示します。

ドライブサイズ（**8+2 RAID 6**）ストレージボリュームグループ	ストレージ容量（ファイル内容）
1.92TB	59.89TB
3.84TB	119.80TB

ドライブサイズ（**8+2 RAID 6**）ストレージボリュームグループ	ストレージ容量（ファイル内容）
7.68TB	251.89TB
15.3TB	538.55TB

グローバルファイルロックが有効になっていないかぎり、ベース（1番目）のビルディングブロックに管理サービスを含める場合のパフォーマンスと容量のオーバーヘッドは最小限です。

パフォーマンスの調整

BeeGFS解決策 には、検証テストに基づいたパフォーマンス調整の推奨事項が含まれます。

BeeGFSは設定不要で妥当なパフォーマンスを提供しますが、パフォーマンスを最大化するために推奨されるチューニングパラメータを一連開発しました。これらのパラメータには、基盤となるEシリーズのブロックノードの機能と、共有ディスクのHAアーキテクチャでBeeGFSを実行するために必要な特別な要件が考慮されています。

ファイルノードのパフォーマンス調整

設定可能なチューニングパラメータには、次のものがあります。

1. *ファイルノードのUEFI/BIOSのシステム設定。*パフォーマンスを最大化するには、ファイルノードとして使用するサーバーモデルのシステム設定を構成することをお勧めします。システム設定は、ベースボード管理コントローラ（BMC）が提供するセットアップユーティリティ（UEFI / BIOS）またはRedfish APIを使用してファイルノードをセットアップするときに設定します。

   システム設定は、ファイルノードとして使用するサーバーモデルによって異なります。使用中のサーバモデルに基づいて、設定を手動で行う必要があります。検証済みLenovo SR665ファイルノードのシステム設定を構成する方法については、を参照してください "パフォーマンスのファイルノードシステム設定を調整します"。

2. *必須構成パラメータのデフォルト設定。*必要な構成パラメータは、BeeGFSサービスの構成方法、およびペースメーカーによるEシリーズボリューム（ブロックデバイス）のフォーマットとマウント方法に影響します。これらの必須設定パラメータには、次のものがあります。

   - BeeGFSサービスの設定パラメータ

     必要に応じて、設定パラメータのデフォルト設定を上書きできます。特定のワークロードやユースケースに合わせて調整できるパラメータについては、を参照してください "BeeGFSサービスの設定パラメータ"。

   - ボリュームのフォーマットとマウントのパラメータは推奨されるデフォルトに設定されており、高度なユースケースでのみ調整する必要があります。デフォルト値では、次の処理が実行されます。

     - ターゲットタイプ（管理、メタデータ、ストレージなど）、基盤となるボリュームのRAID構成とセグメントサイズに基づいて、ボリュームの初期フォーマットを最適化します。

     - Pacemakerが各ボリュームをマウントする方法を調整し、変更がEシリーズのブロックノードにすぐにフラッシュされるようにします。これにより、アクティブな書き込みが進行中の状態でファイルノードに障害が発生してもデータが失われることはありませ

特定のワークロードやユースケースに合わせて調整できるパラメータについては、を参照してください "ボリュームのフォーマットと構成パラメータのマウント"。

3. *ファイルノードにインストールされているLinux OSのシステム設定。*の手順4でAnsibleインベントリを作成すると、デフォルトのLinux OSシステム設定を上書きできます "Ansibleインベントリを作成します"。

   デフォルトの設定を使用してNetApp解決策 のBeeGFSが検証されましたが、特定のワークロードやユースケースに合わせて調整するように変更することができます。変更可能なLinux OSのシステム設定には、次のようなものがあります。

   - Eシリーズのブロックデバイスの I/Oキュー。

     BeeGFSターゲットとして使用されるEシリーズのブロックデバイスでは、次の目的でI/Oキューを設定できます。

     - デバイスタイプ（NVMe、HDDなど）に基づいてスケジューリングアルゴリズムを調整します。
     - 未処理の要求数を増やします。
     - 要求サイズを調整します。
     - 先読み動作を最適化します。
   - 仮想メモリの設定。

     仮想メモリの設定を調整して、最適な持続ストリーミングパフォーマンスを得ることができます。

   - CPU設定。

     CPU周波数ガバナおよびその他のCPU構成を調整して、パフォーマンスを最大限に高めることができます。

   - 読み取り要求のサイズ

     Mellanox HCAの読み取り要求の最大サイズを増やすことができます。

ブロックノードのパフォーマンス調整

特定のBeeGFSビルディングブロックに適用される構成プロファイルに基づいて、ブロックノードに設定されたボリュームグループが少し変化します。たとえば、24ドライブのEF600ブロックノードがある場合、次のようになります。

- BeeGFSの管理、メタデータ、ストレージサービスなど、単一のベースビルディングブロックの場合は次の手順を実行します。
  - BeeGFS管理およびメタデータサービス用に2+2 RAID 10ボリュームグループが1つ
  - BeeGFSストレージサービス用に8+2 RAID 6ボリュームグループが2つ
- BeeGFSメタデータとストレージビルディングブロックの場合：
  - BeeGFSメタデータサービス用に2+2 RAID 10ボリュームグループが1つ
  - BeeGFSストレージサービス用に8+2 RAID 6ボリュームグループが2つ
- BeeGFSストレージのみのビルディングブロックの場合：

- BeeGFSストレージサービス向けに10+2 RAID 6ボリュームグループが2つ

> BeeGFSでは管理とメタデータのストレージ容量がストレージよりも大幅に少ないため、RAID 10ボリュームグループには小さいドライブを使用する方法も1つあります。より小さいドライブは、最も外側のドライブスロットに取り付ける必要があります。詳細については、を参照してください "導入手順"。

これらはすべてAnsibleベースの導入で設定され、次のようなパフォーマンスや動作を最適化するために一般的に推奨されるいくつかの設定とともに使用されます。

- グローバルキャッシュブロックサイズを32KiBに調整し、デマンドベースのキャッシュフラッシュを80%に調整しています。
- 自動ロードバランシングを無効にする（コントローラのボリュームの割り当ては意図したとおりに維持する）。
- 読み取りキャッシュの有効化と先読みキャッシュの無効化
- ミラーリングあり、バッテリバックアップを必要とする書き込みキャッシュを有効にして、ブロックノードコントローラの障害後もキャッシュを維持できるようにします。
- ボリュームグループにドライブを割り当てる順序を指定し、使用可能なドライブチャネル間でI/Oのバランスを調整します。

解決策 を導入します

導入の概要

NetAppのBeeGFSビルディング・ブロック設計の第2世代を使用して、NetApp にBeeGFSを導入し、検証済みのファイル・ノードとブロック・ノードを導入できます。

Ansibleのコレクションとロール

アプリケーションの導入を自動化するために広く使用されているIT自動化エンジンであるAnsibleを使用して、NetApp解決策 にBeeGFSを導入します。Ansibleは、展開するBeeGFSファイルシステムをモデル化するインベントリとしてまとめて呼ばれる一連のファイルを使用します。

Ansibleを使用すると、ネットアップなどの企業は、Ansible Galaxyのコレクションを使用して組み込みの機能を拡張できます（を参照） "NetApp EシリーズBeeGFSのコレクション"）。コレクションには、特定の機能やタスク（Eシリーズボリュームの作成など）を実行するモジュールや、複数のモジュールやその他の役割を呼び出すことができるロールが含まれます。この自動化されたアプローチにより、BeeGFSファイルシステムと基盤となるHAクラスタの導入に要する時間が短縮されます。さらに、ビルディングブロックの追加が簡単になり、既存のファイルシステムを拡張できます。

詳細については、を参照してください "Ansibleのインベントリを確認できます"。

> NetApp解決策 へのBeeGFSの導入には多数の手順が含まれるため、解決策 の手動による導入はサポートされません。

BeeGFSビルディングブロックの構成プロファイル

導入手順では、次の設定プロファイルについて説明します。

- 管理、メタデータ、ストレージサービスを含む1つのベースとなるビルディングブロックです。
- メタデータとストレージサービスを含む2つ目のビルディングブロック。
- ストレージサービスのみを含む3つ目のビルディングブロック。

これらのプロファイルは、NetApp BeeGFSビルディングブロックに推奨されるすべての構成プロファイルを示しています。環境ごとに、メタデータおよびストレージのビルディングブロックまたはストレージサービス専用のビルディングブロックの数は、容量とパフォーマンスの要件に応じて手順によって異なります。

導入手順の概要

の導入では、次の作業を実行します。

ハードウェアの導入

1. 各ビルディングブロックを物理的に組み立てます。
2. ラックに設置してケーブルを配線する。詳細な手順については、を参照してください "ハードウェアを導入"。

ソフトウェアの導入

1. "ファイルノードとブロックノードをセットアップ"。
    - ファイルノードにBMCのIPを設定します
    - サポートされているオペレーティングシステムをインストールし、ファイルノードに管理ネットワークを設定します
    - ブロックノードに管理IPを設定します
2. "Ansibleコントロールノードをセットアップします"。
3. "パフォーマンスのシステム設定を調整します"。
4. "Ansibleインベントリを作成します"。
5. "BeeGFSビルディングブロックのAnsibleインベントリを定義します"。
6. "Ansibleを使用してBeeGFSを導入します"。
7. "BeeGFSクライアントを設定します"。

> 展開手順には、テキストをファイルにコピーする必要があるいくつかの例が含まれています。特定の導入環境で変更する必要があるもの、または変更可能なものについては、「#」または「/」文字で示されるインラインコメントに十分に注意してください。例：'beegfs_ha_ntp_server_pools:#これはコメントの例です-「pool 0.pool.ntp.org iBurst maxsources 3」-「pool 1.pool.ntp.org iBurst maxsources 3」

Ansibleのインベントリを確認できます

導入を開始する前に、Ansibleを使用して、第2世代のBeeGFSビルディングブロック設計を使用してNetApp解決策 にBeeGFSを設定して導入する方法を確認してください。

Ansibleインベントリは、ファイルノードとブロックノードの構成を定義し、導入するBeeGFSファイルシステムを表します。インベントリには、目的のBeeGFSファイルシステムを記述するホスト、グループ、および変数が含まれます。サンプルインベントリは、からダウンロードできます "NetApp EシリーズBeeGFS GitHub"。

Ansibleのモジュールとロール

Ansibleインベントリで説明されている構成を適用するには、NetApp EシリーズAnsibleコレクションに含まれるさまざまなAnsibleモジュールとロール、特にBeeGFS HA 7.2ロール（から入手可能）を使用します "NetApp EシリーズBeeGFS GitHub"）を使用してエンドツーエンドの解決策 を導入します。

NetApp EシリーズAnsibleコレクションの各ロールは、NetApp解決策 上のBeeGFSをエンドツーエンドで導入します。これらのロールでは、NetApp E-Series SANtricity 、Host、およびBeeGFSの各コレクションを使用して、HA（High Availability）を使用してBeeGFSファイルシステムを設定できます。その後、ストレージをプロビジョニングしてマッピングし、クラスタストレージを使用できる状態にします。

ロールには詳細なドキュメントが含まれていますが、導入手順では、第2世代のBeeGFSビルディングブロック設計を使用して、ロールを使用してNetApp Verified Architectureを導入する方法について説明します。

> 導入手順でAnsibleの使用経験が十分に細かい情報を提供できるようにすることは前提条件ではありませんが、Ansibleと関連する用語についてある程度理解している必要があります。

BeeGFS HAクラスタのインベントリレイアウト

Ansibleのインベントリ構造を使用して、BeeGFS HAクラスタを定義します。

以前のAnsibleの経験がある方は、BeeGFS HAロールでは、各ホストに適用される変数（ファクト）を検出するカスタムメソッドが実装されることに注意してください。これは、複数のサーバで実行可能なリソースを記述したAnsibleインベントリの構築を簡易化するために必要です。

Ansibleのインベントリは通常'host_vars'および'group_vars'のファイルと'特定のグループ（および他のグループに潜在的にグループ）にホストを割り当てる'inventory.yml'ファイルで構成されます

> このサブセクションの内容を含むファイルは、例としてのみ作成しないでください。

この構成は構成プロファイルに基づいて事前定義されていますが、以下に示すように、すべてがAnsibleインベントリとしてどのようにレイアウトされるかについて一般的に理解しておく必要があります。

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
        meta_01:  # Group representing a metadata service with ID 01.
          hosts:
            file_node_01:  # This service is preferred on the first file
node.
            file_node_02:  # And can failover to the second file node.
        meta_02:  # Group representing a metadata service with ID 02.
          hosts:
            file_node_02:  # This service is preferred on the second file
node.
            file_node_01: # And can failover to the first file node.
```

サービスごとに'構成を記述するgroup_varsの下に追加ファイルが作成されます

```
# meta_01 - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: 8015
  connMetaPortUDP: 8015
  tuneBindToNumaZone: 0
floating_ips:
  - i1b: <IP>/<SUBNET_MASK>
  - i4b: <IP>/<SUBNET_MASK>
# Type of BeeGFS service the HA resource group will manage.
beegfs_service: metadata # Choices: management, metadata, storage.
# What block node should be used to create a volume for this service:
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        owning_controller: A
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.25
```

このレイアウトでは、各リソースのBeeGFSサービス、ネットワーク、ストレージの構成を1か所で定義できます。バックグラウンドでは、BeeGFSロールは、このインベントリ構造に基づいて各ファイルおよびブロックノードに必要な設定を集約します。詳細については、次のブログ記事を参照してください。 "ネットアップがAnsibleでBeeGFSのHA構成の導入を高速化"。

> 各サービスのBeeGFS数値と文字列ノードIDは、グループ名に基づいて自動的に設定されます。したがって、グループ名を一意にするための一般的なAnsibleの要件に加えて、BeeGFSサービスを表すグループは、グループが表すBeeGFSサービスのタイプに一意の番号で終わる必要があります。たとえば、meta_01とstor_01は許可されますが、meta_01とmeta_01は許可されません。

ベストプラクティスを確認

NetApp解決策 にBeeGFSを導入する際は、ベストプラクティスのガイドラインに従ってください。

標準規則

Ansibleインベントリファイルを物理的に構築して作成する場合は、以下の標準的な規則に従ってください（詳細については、を参照してください） "Ansibleインベントリを作成します"）。

- ファイル・ノードのホスト名は'順番に番号が付けられ（H01-HN）'ラックの一番上に番号が小さい'下に数字が大きい

たとえば'[location][row][rack][hn']という命名規則は'ictad22h01'のようになります

- 各ブロックノードは2台のストレージコントローラで構成され、コントローラごとに独自のホスト名が付けられます。

  ストレージアレイ名は、Ansibleインベントリの一部としてブロックストレージシステム全体を参照するために使用されます。ストレージアレイ名は順番に（A01-AN）番号を付け、各コントローラのホスト名はこの命名規則に基づいて付けます。

  たとえば、「ictad22a01」という名前のブロックノードでは、通常、「ictad22a01-a」や「ictad22a01-b」などの各コントローラにホスト名を設定できますが、Ansibleのインベントリでは「ictad22a01」と呼ばれます。

- 同じビルディングブロック内のファイルノードとブロックノードは、同じ番号方式を共有し、ラック内で互いに隣接しています。一番上に両方のファイルノードがあり、その下に両方のブロックノードが直接配置されています。

  たとえば、最初のビルディングブロックでは、ファイルノードH01とH02がどちらもブロックノードA01とA02に直接接続されています。ホスト名は、H01、H02、A01、およびA02です。

- ビルディングブロックは、ホスト名に基づいて順に配置されるため、番号の小さいホスト名はラックの上部に、番号の大きいホスト名は下部に配置されます。

  ここでは、ケーブルをラックスイッチの上部まで配線する時間を最小限に抑え、トラブルシューティングを簡単にするための標準的な導入方法を定義します。ラックの安定性に問題があるためにこれが許可されないデータセンターでは、逆の場合は、下から上にラックにデータを入力することができます。

InfiniBandストレージネットワーク構成

各ファイルノードの半分のInfiniBandポートをブロックノードに直接接続します。残りの半分はInfiniBandスイッチに接続され、BeeGFSクライアント/サーバ接続に使用されます。BeeGFSクライアントおよびサーバに使用するIPoIBサブネットのサイズを決定する際には、コンピューティング/ GPUクラスタとBeeGFSファイルシステムの予想される増加を考慮する必要があります。推奨されるIP範囲から外れる必要がある場合は、1つのビルディングブロック内の各直接接続に一意のサブネットがあり、クライアント/サーバ接続に使用されるサブネットと重複しないことに注意してください。

直接接続

各ビルディングブロック内のファイルノードとブロックノードは、常に次の表のIPを使用して直接接続されます。

> このアドレス指定方式は、次のルールに従っています。3番目のオクテットは常に奇数または偶数で、ファイルノードが奇数であるか偶数であるかによって異なります。

ファイルノード	**IB**ポート	**IP** アドレス	ブロックノード	**IB**ポート	物理**IP**	仮想**IP**
奇数（h1）	i1a	192.168.1.10	奇数（c1）	2A	192.168.1.100	192.168.1.101
奇数（h1）	i2a	192.168.3.10	奇数（c1）	2A	192.168.3.100	192.168.3.101
奇数（h1）	i3a	192.168.5.10	偶数（C2）	2A	192.168.5.100	192.168.5.101

ファイルノード	**IB**ポート	**IP** アドレス	ブロックノード	**IB**ポート	物理**IP**	仮想**IP**
奇数（h1）	i4a	192.168.7.10	偶数（C2）	2A	192.168.7.100	192.168.7.101
偶数（h2）	i1a	192.168.2.10	奇数（c1）	2B	192.168.2.100	192.168.2.101
偶数（h2）	i2a	192.168.4.10	奇数（c1）	2B	192.168.4.100	192.168.4.101
偶数（h2）	i3a	192.168.6.10	偶数（C2）	2B	192.168.6.100	192.168.6.101
偶数（h2）	i4a	192.168.8.10	偶数（C2）	2B	192.168.8.100	192.168.8.101

BeeGFSクライアント**/**サーバ**IPoIB**アドレッシング方式（**2**つのサブネット）

BeeGFSクライアントが2つのInfiniBandポートを使用できるようにするには、各サブネットで優先IPを設定したBeeGFSサーバサービスの半分に2つのIPoIBサブネットを設定して、クライアントが2つのInfiniBandポートを使用してファイルシステムの冗長性とスループットを最大化する必要があります。

各ファイルノードは、複数のBeeGFSサーバサービス（管理、メタデータ、またはストレージ）を実行します。各サービスを他のファイルノードに個別にフェイルオーバーできるようにするために、各サービスには両方のノード間で共有できる一意のIPアドレス（論理インターフェイスまたはLIFとも呼ばれます）が設定されます。

必須ではありませんが、この配置では、次のIPoIBサブネット範囲がこれらの接続に使用されていることが前提となり、次の規則を適用する標準アドレッシング方式を定義します。

- 2番目のオクテットは、ファイルノードのInfiniBandポートが奇数であるか偶数であるかに基づいて常に奇数になります。
- BeeGFSクラスタIPは常にxxxです。127.100.yyyまたは’xxx.128.100.yyy

> インバンドOS管理に使用されるインターフェイスに加えて、Corosyncでクラスタのハートビートおよび同期に追加のインターフェイスを使用できます。これにより、1つのインターフェイスが停止してもクラスタ全体が停止することはありません。

- BeeGFS管理サービスは’常に’xxx.yyy.101.0’または’xxx.yyy.102.0`になります
- BeeGFSメタデータ・サービスは常に「xxx.yyy.101.zzz」または「xxx.yyy.102.zzz」です。
- BeeGFSストレージ・サービスは’常にxxx.103.zzzまたは’xxx.yyy.103.zzz’です
- アドレス範囲は'100.xxx.1.1'〜'100.xxx.99.255’で’クライアント用に予約されています

次の表に、サブネットAの範囲を示します。100.127.0.0.0/16

目的	**InfiniBand**ポート	**IP**アドレスまたは**IP**範囲
BeeGFS Cluster IP（BeeGFSクラスタIP）	i1b	100.127.100.1-100.127.100.255
BeeGFS Managementの略	i1b	100.127.101.0
BeeGFSメタデータ	i1bまたはi3b	100.127.101.1-100.127.101.255
BeeGFS Storage（BeeGFSストレージ）	i1bまたはi3b	100.127.103.1-100.127.103.255

目的	**InfiniBand**ポート	**IP**アドレスまたは**IP**範囲
BeeGFSクライアント	（クライアントによって異なる）	100.127.1.1～100.127.99.255

次の表に、サブネットBの範囲を示します。100.128.0.0/16

目的	**InfiniBand**ポート	**IP**アドレスまたは**IP**範囲
BeeGFS Cluster IP（BeeGFSクラスタIP）	i4b	100.128.100.1~100.128.100.255
BeeGFS Managementの略	i2b	100.128.102.0
BeeGFSメタデータ	i2bまたはi4b	100.128.102.1～100.128.102.255
BeeGFS Storage（BeeGFSストレージ）	i2bまたはi4b	100.128.104.1～100.128.104.255
BeeGFSクライアント	（クライアントによって異なる）	100.128.1.1～100.128.99.255

このNetApp Verified Architectureでは、上記の範囲のすべてのIPが使用されているわけではありません。一貫したIPアドレッシング方式を使用してファイルシステムを簡単に拡張できるように、IPアドレスを事前に割り当てる方法を示します。この方式では、BeeGFSファイルノードとサービスIDは既知のIP範囲の4番目のオクテットに対応します。ファイルシステムは、必要に応じて255ノード以上のノードやサービスを拡張できます。

ハードウェアを導入

各ビルディングブロックは、HDR（200GB）InfiniBandケーブルを使用して2つのブロックノードに直接接続された、検証済みの2つのx86ファイルノードで構成されます。

各ビルディングブロックには2つのBeeGFSファイルノードが含まれるため、フェイルオーバークラスタでクォーラムを確立するには少なくとも2つのビルディングブロックが必要です。2ノードクラスタを構成することは可能ですが、この構成には制限があり、場合によってはフェイルオーバーを正常に実行できません。2ノードクラスタが必要な場合は、この導入手順 では説明していませんが、3つ目のデバイスをTiebreakerとして組み込むこともできます。

特に明記しないかぎり、BeeGFSメタデータとストレージサービスまたはストレージサービスのみの実行に使用するかどうかにかかわらず、次の手順はクラスタ内の各ビルディングブロックで同じです。

手順

1. 4つのPCIe 4.0 ConnectX-6デュアルポートホストチャネルアダプタ（HCA）を使用して各BeeGFSファイルノードをInfiniBandモードで設定し、PCIeスロット2、3、5、6に取り付けます。
2. デュアルポートの200GBホストインターフェイスカード（HIC）で各BeeGFSブロックノードを設定し、2台の各ストレージコントローラにHICを取り付けます。

   2つのBeeGFSファイルノードがBeeGFSブロックノードの上になるようにビルディングブロックをラックに配置します。次の図は、BeeGFSビルディングブロックの正しいハードウェア構成を示しています（背面図）。

   []

一般に、本番環境では電源装置を冗長PSUにする必要があります。

3. 必要に応じて、BeeGFSブロックノードのそれぞれにドライブを取り付けます。
   a. ビルディングブロックを使用してBeeGFSメタデータとストレージサービスを実行し、さらに小さいドライブをメタデータボリュームに使用する場合は、次の図に示すように、最も外側のドライブスロットにそれらが搭載されていることを確認します。
   b. すべてのビルディングブロック構成で、ドライブエンクロージャにフル装備されていない場合は、最適なパフォーマンスを得るために、同じ数のドライブがスロット0_{11}および$_{12}$23に装着されていることを確認してください。

      []

4. ファイルノードとブロックノードをケーブル接続するには、次の図に示すトポロジと一致するように、1 m InfiniBand HDR 200GBの直接接続銅ケーブルを使用します。

   []

   複数のビルディングブロックを横断するノードが直接接続されることはありません。各ビルディングブロックはスタンドアロンユニットとして扱われ、ビルディングブロック間のすべての通信はネットワークスイッチを介して行われます。

5. 2m（または適切な長さ）InfiniBand HDR 200GB直接接続銅ケーブルを使用して、各ファイルノードの残りのInfiniBandポートをストレージネットワークに使用するInfiniBandスイッチにケーブル接続します。

   使用中の冗長InfiniBandスイッチがある場合は、次の図の薄い緑でハイライトされたポートを別々のスイッチにケーブル接続します。

   []

6. 必要に応じて、同じケーブル配線ガイドラインに従って追加のビルディングブロックをアセンブルします。

   1台のラックに導入できるビルディングブロックの総数は、各サイトで利用可能な電力と冷却量によって異なります。

ソフトウェアを導入

ファイルノードとブロックノードをセットアップ

ほとんどのソフトウェア設定タスクはネットアップが提供するAnsibleコレクションを使用して自動化されていますが、各サーバのBaseboard Management Controller（BMC；ベースボード管理コントローラ）でネットワークを設定し、各コントローラの管理ポートを設定する必要があります。

ファイルノードをセットアップします

1. 各サーバのベースボード管理コントローラ（BMC）のネットワークを設定します。

検証済みLenovo SR665ファイルノードのネットワークを設定する方法については、を参照してください "Lenovo ThinkSystemのドキュメント"。

> ベースボード管理コントローラ（BMC）は、サービスプロセッサとも呼ばれ、オペレーティングシステムがインストールされていない場合やアクセスできない場合でもリモートアクセスを提供できるさまざまなサーバプラットフォームに組み込まれているアウトオブバンド管理機能の一般的な名前です。ベンダーは通常、この機能を独自のブランドで販売しています。たとえば、Lenovo SR665では、BMCは_Lenovo XClarity Controller（XCC）_と呼ばれています。

2. 最大のパフォーマンスを得るためにシステムを設定します。

   システム設定は、UEFIセットアップ（旧BIOS）を使用するか、多くのBMCが提供するRedfish APIを使用して設定します。システム設定は、ファイルノードとして使用するサーバモデルによって異なります。

   検証済みLenovo SR665ファイルノードのシステム設定を構成する方法については、を参照してください "パフォーマンスのシステム設定を調整します"。

3. Red Hat 8.4をインストールし、AnsibleコントロールノードからのSSH接続など、オペレーティングシステムの管理に使用するホスト名とネットワークポートを設定します。

   この時点では、InfiniBandポートにIPを設定しないでください。

   > 厳密には必須ではありませんが、以降のセクションでは、ホスト名には順番に番号が付けられ（h1-hNなど）、奇数ホストと偶数ホストで完了する必要のあるタスクを参照するようにしています。

4. Red Hatの公式リポジトリから必要なパッケージをインストールできるようにしたり、Red Hatのサポートされているバージョンに対する更新を制限するには、Red Hat Subscription Managerを使用してシステムを登録およびサブスクライブします。手順については、を参照してください "RHELシステムを登録および登録する方法" および "更新を制限する方法"。

5. ハイアベイラビリティに必要なパッケージを含むRed Hatリポジトリを有効にします。

   ```
   subscription-manager repo-override --repo=rhel-8-for-x86_64
   -highavailability-rpms --add=enabled:1
   ```

6. すべてのConnectX-6 HCAファームウェアを、で推奨されているバージョンにアップデートします "テクノロジ要件"。

   この更新を行うには'推奨ファームウェアをバンドルするmlxupツールのバージョンをダウンロードして実行しますこのツールはからダウンロードできます "mlxup -アップデートとクエリユーティリティ"。

ブロックノードをセットアップする

各コントローラの管理ポートを設定してEF600ブロックノードをセットアップします。

1. 各EF600コントローラに管理ポートを設定します。

   ポートの設定手順については、を参照してください "Eシリーズドキュメントセンター"。

2. 必要に応じて、各システムのストレージアレイ名を設定します。

   名前を設定すると、以降のセクションで各システムを簡単に参照できるようになります。アレイ名の設定手順については、を参照してください "Eシリーズドキュメントセンター"。

> 必須ではありませんが、以降のトピックでは、ストレージアレイ名には必ず順番に番号を付け（c1-CNなど）、奇数のシステムでも偶数のシステムでも完了する必要がある手順を参照してください。

Ansibleコントロールノードをセットアップします

Ansible制御ノードをセットアップするには、解決策 の設定に使用できるすべてのファイルノードとブロックノードの管理ポートへのネットワークアクセスがある仮想マシンまたは物理マシンを特定する必要があります。

次の手順は、CentOS 8.4でテストしました。使用するLinuxディストリビューションに固有の手順については、を参照してください "Ansibleのドキュメント"。

1. Python 3.9をインストールし、正しいバージョンの「pip」がインストールされていることを確認します。

```
sudo dnf install python3.9 -y
sudo dnf install python39-pip
sudo dnf install sshpass
```

2. Python 3.9バイナリが'python3'または'python'が呼び出されるたびに使用されるように'シンボリック・リンクを作成します

```
sudo ln -sf /usr/bin/python3.9 /usr/bin/python3
sudo ln -sf /usr/bin/python3 /usr/bin/python
```

3. NetApp BeeGFSコレクションに必要なPythonパッケージをインストールします。

```
python3 -m pip install ansible cryptography netaddr
```

> サポートされているバージョンのAnsibleと必要なすべてのPythonパッケージをインストールするには、BeeGFSコレクションのReadmeファイルを参照してください。サポートされているバージョンについても、で説明します "技術要件"。

4. 正しいバージョンのAnsibleとPythonがインストールされていることを確認します。

```
ansible --version
ansible [core 2.11.6]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.9/site-
packages/ansible
  ansible collection location =
/root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/local/bin/ansible
  python version = 3.9.2 (default, Mar 10 2021, 17:29:56) [GCC 8.4.1
20200928 (Red Hat 8.4.1-1)]
  jinja version = 3.0.2
  libyaml = True
```

5. BeeGFSの導入を表すために使用するAnsibleインベントリをGitやBitBucketなどのソース管理システムに保存し、Gitをインストールしてこれらのシステムと対話します。

```
sudo dnf install git -y
```

6. パスワードを使用しないSSHを設定します。これは、AnsibleがAnsibleの制御ノードからリモートのBeeGFSファイルノードにアクセスできるようにする最も簡単な方法です。
   a. 必要に応じて、Ansibleのコントロールノードで、「ssh-keygen」を使用して公開鍵のペアを生成します
   b. 「ssh-copy-id <IP_or_hostname>」を使用して、各ファイルノードにパスワードなしSSHを設定します

      ブロックノードにパスワードなしのSSHを設定しないでください。サポートされていません。

7. に示すBeeGFSコレクションのバージョンをインストールするには、Ansible Galaxyを使用します "技術要件"。

   このインストールには、NetApp SANtricity ソフトウェアやホストコレクションなどのAnsibleの依存関係も含まれます。

```
ansible-galaxy collection install netapp_eseries.beegfs:==3.0.1
```

Ansibleインベントリを作成します

ファイルノードとブロックノードの設定を定義するには、導入するBeeGFSファイルシステムを表すAnsibleインベントリを作成します。インベントリには、目的のBeeGFSファイルシステムを記述するホスト、グループ、および変数が含まれます。

手順**1**：すべてのビルディングブロックの構成を定義します

どの構成プロファイルを個別に適用できるかに関係なく、環境 のすべての構成ブロックを定義します。

作業を開始する前に

- BitBucketやGitなどのソース管理システムを使用して、AnsibleのインベントリファイルとPlaybookファイルが格納されているディレクトリのコンテンツを保存します。
- Gitが無視するファイルを指定する「.gitignore」ファイルを作成します。これにより、Gitに大きなファイルを保存することを回避できます。

手順

1. Ansibleの制御ノードで、Ansibleのインベントリファイルとプレイブックファイルの格納に使用するディレクトリを特定します。

   特に記載がないかぎり、この手順および以降の手順で作成するすべてのファイルとディレクトリは、このディレクトリを基準にして作成されます。

2. 次のサブディレクトリを作成します。

   「host_vars」

   'group_vars'

   「パッケージ」

手順**2**：個々のファイルノードとブロックノードの設定を定義する

環境 の個々のファイルノードおよび個々のビルディングブロックノードの構成を定義します。

1. 「host_vars/`」で、「<hostname>.yml」という名前のBeeGFSファイルノードごとに次の内容のファイルを作成します。BeeGFSクラスタのIPおよび奇数で終わるホスト名と偶数で終わるホスト名には、内容に関する注意を払って入力してください。

   最初は、ファイルノードのインターフェイス名が、ここに記載されている名前と一致しています（ib0やibs1f0など）。これらのカスタム名は、で設定します [Step 4: Define configuration that should apply to all file nodes]。

```
ansible_host: “<MANAGEMENT_IP>”
eseries_ipoib_interfaces:  # Used to configure BeeGFS cluster IP
addresses.
  - name: i1b
    address: 100.127.100. <NUMBER_FROM_HOSTNAME>/16
  - name: i4b
    address: 100.128.100. <NUMBER_FROM_HOSTNAME>/16
beegfs_ha_cluster_node_ips:
  - <MANAGEMENT_IP>
  - <i1b_BEEGFS_CLUSTER_IP>
  - <i4b_BEEGFS_CLUSTER_IP>
# NVMe over InfiniBand storage communication protocol information
# For odd numbered file nodes (i.e., h01, h03, ..):
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.1.10/24
    configure: true
  - name: i2a
    address: 192.168.3.10/24
    configure: true
  - name: i3a
    address: 192.168.5.10/24
    configure: true
  - name: i4a
    address: 192.168.7.10/24
    configure: true
# For even numbered file nodes (i.e., h02, h04, ..):
# NVMe over InfiniBand storage communication protocol information
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.2.10/24
    configure: true
  - name: i2a
    address: 192.168.4.10/24
    configure: true
  - name: i3a
    address: 192.168.6.10/24
    configure: true
  - name: i4a
    address: 192.168.8.10/24
    configure: true
```

BeeGFSクラスタをすでに導入している場合は、静的に設定されたIPアドレス（NVMe/IBで使用するクラスタIPやIPなど）を追加または変更する前に、クラスタを停止する必要があります。これは、変更が適切に反映され、クラスタの処理が中断されないようにするために必要です。

2. 「host_vars/`」で、「<hostname>.yml」という名前のBeeGFSブロックノードごとにファイルを作成し、次の内容を入力します。

   ストレージアレイ名の末尾が奇数で偶数である場合は、内容に特に注意してください。

   ブロックノードごとに1つのファイルを作成し、2つのコントローラ（通常はA）のうちの1つに「<MANAGEMENT _IP>」を指定します。

```
eseries_system_name: <STORAGE_ARRAY_NAME>
eseries_system_api_url: https://<MANAGEMENT_IP>:8443/devmgr/v2/
eseries_initiator_protocol: nvme_ib
# For odd numbered block nodes (i.e., a01, a03, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.1.101
    - 192.168.2.101
    - 192.168.1.100
    - 192.168.2.100
  controller_b:
    - 192.168.3.101
    - 192.168.4.101
    - 192.168.3.100
    - 192.168.4.100
# For even numbered block nodes (i.e., a02, a04, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.5.101
    - 192.168.6.101
    - 192.168.5.100
    - 192.168.6.100
  controller_b:
    - 192.168.7.101
    - 192.168.8.101
    - 192.168.7.100
    - 192.168.8.100
```

手順**3**：すべてのファイルノードとブロックノードに適用する設定を定義する

グループに対応するファイル名に'GROLE_vars'の下にあるホストのグループに共通する構成を定義できます
これにより、複数の場所で共有設定を繰り返す必要がなくなります。

ホストは複数のグループに含めることができ、実行時に、Ansibleは、変数の優先順位ルールに基づいて、特定のホストに適用する変数を選択します。（これらのルールの詳細については、Ansibleのドキュメントを参照してください "変数を使用します". ）

ホストとグループの割り当ては、実際のAnsibleインベントリファイルに定義されます。このファイルは、この手順 の末尾に作成されます。

Ansibleでは、すべてのホストに適用する構成は「all」というグループで定義できます。次の内容で'ファイル'group_vars/all.yml'を作成します

```
ansible_python_interpreter: /usr/bin/python3
beegfs_ha_ntp_server_pools:  # Modify the NTP server addressess if
desired.
  - "pool 0.pool.ntp.org iburst maxsources 3"
  - "pool 1.pool.ntp.org iburst maxsources 3"
```

手順**4**：すべてのファイルノードに適用する設定を定義する

ファイル・ノードの共有構成は'ha_cluster'というグループで定義されますこのセクションの手順では'group_vars/ha_cluster.yml`ファイルに含める必要がある構成を構築します

手順

1. ファイルの最上部で'ファイルノードのsudoユーザーとして使用するパスワードを含むデフォルトを定義します

```
### ha_cluster Ansible group inventory file.
# Place all default/common variables for BeeGFS HA cluster resources
below.
### Cluster node defaults
ansible_ssh_user: root
ansible_become_password: <PASSWORD>
eseries_ipoib_default_hook_templates:
  - 99-multihoming.j2 # This is required when configuring additional
static IPs (for example cluster IPs) when multiple IB ports are in the
same IPoIB subnet.
# If the following options are specified, then Ansible will
automatically reboot nodes when necessary for changes to take effect:
eseries_common_allow_host_reboot: true
eseries_common_reboot_test_command: "systemctl --state=active,exited |
grep eseries_nvme_ib.service"
```

> 特に本番環境では、パスワードをプレーンテキストで保存しないでください。代わりにAnsible Vaultを使用します（を参照） "Ansible Vaultを使用したコンテンツの暗号化")または'--Ask -bece-pass`オプションを使用してプレイブックを作成します「Ansible」ssh_user がすでに「root」である場合は、オプションで「Ansibleの_ bece_password」を省略できます。

2. 必要に応じて、ハイアベイラビリティ（HA）クラスタの名前を設定し、クラスタ内通信用のユーザを指定します。

   プライベートIPアドレッシング方式を変更する場合は、デフォルトの「beegfs_ha_mgmtd_floating_ip」も更新する必要があります。これは、後でBeeGFS Managementリソースグループに設定する内容と一致している必要があります。

   「beegfs_alert_email_list」を使用して、クラスタ・イベントのアラートを受信する電子メールを1つ以上指定します。

```
### Cluster information
beegfs_ha_firewall_configure: True
eseries_beegfs_ha_disable_selinux: True
eseries_selinux_state: disabled
# The following variables should be adjusted depending on the desired
configuration:
beegfs_ha_cluster_name: hacluster                   # BeeGFS HA cluster
name.
beegfs_ha_cluster_username: hacluster               # BeeGFS HA cluster
username.
beegfs_ha_cluster_password: hapassword              # BeeGFS HA cluster
username's password.
beegfs_ha_cluster_password_sha512_salt: randomSalt # BeeGFS HA cluster
username's password salt.
beegfs_ha_mgmtd_floating_ip: 100.127.101.0          # BeeGFS management
service IP address.
# Email Alerts Configuration
beegfs_ha_enable_alerts: True
beegfs_ha_alert_email_list: ["email@example.com"]  # E-mail recipient
list for notifications when BeeGFS HA resources change or fail.  Often a
distribution list for the team responsible for managing the cluster.
beegfs_ha_alert_conf_ha_group_options:
      mydomain: “example.com”
# The mydomain parameter specifies the local internet domain name. This
is optional when the cluster nodes have fully qualified hostnames (i.e.
host.example.com).
# Adjusting the following parameters is optional:
beegfs_ha_alert_timestamp_format: "%Y-%m-%d %H:%M:%S.%N" #%H:%M:%S.%N
beegfs_ha_alert_verbosity: 3
#  1) high-level node activity
#  3) high-level node activity + fencing action information + resources
(filter on X-monitor)
#  5) high-level node activity + fencing action information + resources
```

一見冗長に見えても’beegfs_ha_gmtd_floating_ip’は'1つのHAクラスタを超えてBeeGFSファイルシステムを拡張する場合に重要です以降のHAクラスタは、BeeGFS管理サービスを追加せずに導入され、最初のクラスタが提供する管理サービスをポイントします。

3. フェンシングエージェントを設定します。（詳細については、を参照してください "Red Hatハイアベイラビリティクラスタでフェンシングを設定します".） 次の出力は、一般的なフェンシングエージェントを設定する例を示しています。次のいずれかのオプションを選択します。

   この手順では、次の点に注意してください。

   - フェンシングはデフォルトで有効になっていますが、フェンシングエージェント_を設定する必要があります。
   - 'pcmk_host_map’または’pcmk_host_listに指定されている`<hostname>は’Ansibleインベントリ内のホスト名に対応している必要があります
   - フェンシングなしでBeeGFSクラスタを実行することは、特に本番環境ではサポートされません。これは、ブロックデバイスなどのリソース依存関係を含むBeeGFSサービスが問題 によってフェイルオーバーする際に、ファイルシステムの破損やその他の望ましくない動作や予期しない動作を引き起こす複数のノードによる同時アクセスのリスクがないことを主に保証するためです。フェンシングを無効にする必要がある場合は’BeeGFS HAロールの入門ガイドの一般的な注意事項を参照して’ha_cluster.ymlで’beegfs_cluster_crm_config_options[stonith -enabled "]をfalseに設定します
   - 複数のノードレベルのフェンシングデバイスがあり、BeeGFS HAロールでは、Red Hat HAパッケージリポジトリで使用可能なフェンシングエージェントを設定できます。可能な場合は、無停電電源装置（UPS）またはラック配電装置（rPDU）を経由するフェンシングエージェントを使用します。 ベースボード管理コントローラ（BMC）などの一部のフェンシングエージェントや、サーバに組み込まれているその他のライトアウトデバイスは、特定の障害シナリオではフェンス要求に応答しない場合があります。

```
### Fencing configuration:
# OPTION 1: To enable fencing using APC Power Distribution Units
(PDUs):
beegfs_ha_fencing_agents:
 fence_apc:
   - ipaddr: <PDU_IP_ADDRESS>
     login: <PDU_USERNAME>
     passwd: <PDU_PASSWORD>
     pcmk_host_map:
"<HOSTNAME>:<PDU_PORT>,<PDU_PORT>;<HOSTNAME>:<PDU_PORT>,<PDU_PORT>"
# OPTION 2: To enable fencing using the Redfish APIs provided by the
Lenovo XCC (and other BMCs):
redfish: &redfish
  username: <BMC_USERNAME>
  password: <BMC_PASSWORD>
  ssl_insecure: 1 # If a valid SSL certificate is not available
specify “1”.
beegfs_ha_fencing_agents:
  fence_redfish:
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
# For details on configuring other fencing agents see
https://access.redhat.com/documentation/en-
us/red_hat_enterprise_linux/8/html/configuring_and_managing_high_avai
lability_clusters/assembly_configuring-fencing-configuring-and-
managing-high-availability-clusters.
```

4. Linux OSで推奨されるパフォーマンス調整を有効にします。

   多くのユーザはパフォーマンスパラメータのデフォルト設定を確認できますが、特定のワークロードのデフォルト設定は必要に応じて変更できます。そのため、これらの推奨事項はBeeGFSロールに含まれますが、デフォルトでは有効になっていないため、ユーザーはファイルシステムに適用された調整を認識できません。

   パフォーマンス・チューニングを有効にするには’次のように指定

   ```
   ### Performance Configuration:
   beegfs_ha_enable_performance_tuning: True
   ```

5. （オプション）Linux OSのパフォーマンス調整パラメータを必要に応じて調整できます。

調整可能なチューニングパラメータの一覧については、のBeeGFS HAロールの「Performance Tuning Defaults」セクションを参照してください "EシリーズBeeGFS GitHubサイト"。このファイル内のクラスタ内のすべてのノードまたは個々のノードの'host_vars'ファイルで'デフォルト値を上書きできます

6. ブロックノードとファイルノードの間に200Gb / HDRを完全に接続できるようにするには、Mellanox Open Fabrics Enterprise Distribution（MLNx_OFED）のOpen Subnet Manager（OpenSM）パッケージを使用します。(受信箱openssl'パッケージは必要な仮想化機能をサポートしていません) Ansibleによる導入もサポートされていますが、最初に目的のパッケージをBeeGFSロールの実行に使用するAnsibleコントロールノードにダウンロードする必要があります。

   a. 「curl」または希望のツールを使用して、MellanoxのWebサイトの技術要件セクションに記載されているOpenSMのバージョン用のパッケージを「packages/」ディレクトリにダウンロードします。例：

```
curl -o packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm

curl -o packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
```

   b. 'group_vars/ha_cluster.yml'の次のパラメータを入力します(必要に応じてパッケージを調整します)

```
### OpenSM package and configuration information
eseries_ib_opensm_allow_upgrades: true
eseries_ib_opensm_skip_package_validation: true
eseries_ib_opensm_rhel_packages: []
eseries_ib_opensm_custom_packages:
  install:
    - files:
        add:
          "packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
          "packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
    - packages:
        add:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
  uninstall:
    - packages:
        remove:
          - opensm
          - opensm-libs
      files:
        remove:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
eseries_ib_opensm_options:
  virt_enabled: "2"
```

7. 論理InfiniBandポート識別子と基盤となるPCIeデバイスとのマッピングが一貫して行われるように'udev'ルールを設定します

   udevルールは'BeeGFSファイル・ノードとして使用される各サーバ・プラットフォームのPCIeトポロジーに固有のものである必要があります

   検証済みファイルノードには、次の値を使用します。

```
### Ensure Consistent Logical IB Port Numbering
# OPTION 1: Lenovo SR665 PCIe address-to-logical IB port mapping:
eseries_ipoib_udev_rules:
  "0000:41:00.0": i1a
  "0000:41:00.1": i1b
  "0000:01:00.0": i2a
  "0000:01:00.1": i2b
  "0000:a1:00.0": i3a
  "0000:a1:00.1": i3b
  "0000:81:00.0": i4a
  "0000:81:00.1": i4b

# Note: At this time no other x86 servers have been qualified.
Configuration for future qualified file nodes will be added here.
```

8. （オプション）メタデータターゲット選択アルゴリズムを更新します。

```
beegfs_ha_beegfs_meta_conf_ha_group_options:
  tuneTargetChooser: randomrobin
```

> 検証テストでは'通常'randomrobinを使用して'パフォーマンス・ベンチマーク中にテスト・ファイルがすべてのBeeGFSストレージ・ターゲットに均等に分散されるようにしました（ベンチマークの詳細については'BeeGFSのサイトを参照してください "BeeGFSシステムのベンチマーク"）。実際に使用されている場合は、原因 の番号が小さいターゲットが、番号の大きいターゲットよりも早くいっぱいになる可能性があります。「randomrobin」を省略し、デフォルトの「randomized」値を使用するだけで、利用可能なすべてのターゲットを利用しながら、優れたパフォーマンスを提供できるようになりました。

手順**5**：共通ブロックノードの設定を定義する

ブロック・ノードの共有構成は'eseries_storage_systems'というグループで定義されますこのセクションの手順では'group_vars/eseries_storage_systems.yml`ファイルに含める必要がある構成を構築します

手順

1. Ansible接続をローカルに設定し、システムパスワードを指定して、SSL証明書を検証するかどうかを指定します。（通常、AnsibleはSSHを使用して管理対象ホストに接続しますが、NetApp Eシリーズストレージシステムがブロックノードとして使用されている場合、モジュールはREST APIを使用して通信します）。 ファイルの上部に、次の情報を追加します。

```
### eseries_storage_systems Ansible group inventory file.
# Place all default/common variables for NetApp E-Series Storage Systems
here:
ansible_connection: local
eseries_system_password: <PASSWORD>
eseries_validate_certs: false
```

> プレーンテキストでパスワードを一覧表示することは推奨されません。--extra-bvarsを使用してAnsibleを実行するときに'Ansibleボールトを使用するか'eseries_system_password'を提供します

2. 最適なパフォーマンスを確保するには、に記載されているバージョンをブロックノードにインストールします "技術要件"。

   対応するファイルをからダウンロードします "ネットアップサポートサイト"。これらを手動でアップグレードするか'Ansibleコントロール・ノードのパッケージ/ディレクトリに含めてから'eseries_storage_systemesyml'に以下のパラメータを入力して'Ansibleを使用してアップグレードできます

```
# Firmware, NVSRAM, and Drive Firmware (modify the filenames as needed):
eseries_firmware_firmware: "packages/RCB_11.70.2_6000_61b1131d.dlp"
eseries_firmware_nvsram: "packages/N6000-872834-D06.dlp"
```

3. ブロックノードに取り付けられたドライブに対応する最新のドライブファームウェアをからダウンロードしてインストールします "ネットアップサポートサイト"。これらを手動でアップグレードするか'Ansibleコントロール・ノードのパッケージ/ディレクトリに含めてから'eseries_storage_systemesyml'に以下のパラメータを入力して'Ansibleを使用してアップグレードできます

```
eseries_drive_firmware_firmware_list:
  - "packages/<FILENAME>.dlp"
eseries_drive_firmware_upgrade_drives_online: true
```

> eseries_drive_firmware_upgrade_drivesonlineを'false'に設定すると'アップグレードが高速化されますが'BeeGFSが導入されるまでは実行しないでくださいこれは、アプリケーションエラーを回避するために、アップグレード前にドライブへのすべてのI/Oを停止する必要があるためです。ボリュームを構成する前にオンライン・ドライブ・ファームウェア・アップグレードを実行しても問題が発生しないようにするには'この値を常にtrueに設定することを推奨します

4. パフォーマンスを最適化するには、グローバル構成に対して次の変更を行います。

```
# Global Configuration Defaults
eseries_system_cache_block_size: 32768
eseries_system_cache_flush_threshold: 80
eseries_system_default_host_type: linux dm-mp
eseries_system_autoload_balance: disabled
eseries_system_host_connectivity_reporting: disabled
eseries_system_controller_shelf_id: 99 # Required.
```

5. ボリュームのプロビジョニングと動作を最適化するには、次のパラメータを指定します。

```
# Storage Provisioning Defaults
eseries_volume_size_unit: pct
eseries_volume_read_cache_enable: true
eseries_volume_read_ahead_enable: false
eseries_volume_write_cache_enable: true
eseries_volume_write_cache_mirror_enable: true
eseries_volume_cache_without_batteries: false
eseries_storage_pool_usable_drives:
"99:0,99:23,99:1,99:22,99:2,99:21,99:3,99:20,99:4,99:19,99:5,99:18,99:6,
99:17,99:7,99:16,99:8,99:15,99:9,99:14,99:10,99:13,99:11,99:12"
```

> 「eseries_storage_pool_usable_drives」に指定する値はNetApp EF600ブロックノードに固有であり、新しいボリュームグループにドライブを割り当てる順序を制御します。この順序により、各グループへのI/Oがバックエンドドライブチャネル間で均等に分散されます。

BeeGFSビルディングブロックのAnsibleインベントリを定義します

一般的なAnsibleのインベントリ構造を定義したら、BeeGFSファイルシステムの各ビルディングブロックの設定を定義します。

導入手順では、管理、メタデータ、ストレージサービスなどの基本ビルディングブロックで構成されるファイルシステム、メタデータとストレージサービスを提供する2つ目のビルディングブロック、およびストレージ専用の3つ目のビルディングブロックで構成されるファイルシステムの導入方法を示します。

以下の手順は、BeeGFSファイルシステム全体の要件を満たすようにNetApp BeeGFSビルディングブロックを設定する際に使用する代表的な構成プロファイルをすべて示しています。

> このセクションと以降のセクションで、必要に応じて調整して、導入するBeeGFSファイルシステムを表すインベントリを作成します。特に、各ブロックまたはファイルノードを表すAnsibleホスト名と、ストレージネットワークに必要なIPアドレス指定方式を使用して、BeeGFSファイルノードとクライアントの数に合わせて拡張できます。

手順**1**：**Ansible**インベントリファイルを作成する

手順

1. 新しい’inventory.yml`ファイルを作成し’以下のパラメータを挿入します配置されたブロック・ノードを表すために’必要に応じて’eseries_storage_systems’の下のホストを置き換えますこれらの名前は’host_vars/<filename>.yml`に使用する名前に対応していなければなりません

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
```

以降のセクションでは、「ha_cluster」の下に、クラスタで実行するBeeGFSサービスを表すAnsibleグループを追加で作成します。

手順**2**：管理、メタデータ、ストレージのビルディングブロックのインベントリを設定する

クラスタまたはベースビルディングブロックの最初のビルディングブロックには、メタデータサービスおよびストレージサービスとともにBeeGFS管理サービスが含まれている必要があります。

手順

1. 'inventory.yml’で’ha_cluster: children’の下に次のパラメータを入力します

```
      # ictad22h01/ictad22h02 HA Pair (mgmt/meta/storage building
block):
        mgmt:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_01:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_01:
          hosts:
```

```
            ictad22h01:
            ictad22h02:
        meta_02:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_02:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_03:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_03:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_04:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_04:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_05:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_05:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_06:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_06:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_07:
          hosts:
            ictad22h02:
            ictad22h01:
```

```
        stor_07:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_08:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_08:
          hosts:
            ictad22h02:
            ictad22h01:
```

2. ファイル'group_vars/mgmt.yml'を作成し'以下を含めます

```
# mgmt - BeeGFS HA Management Resource Group
# OPTIONAL: Override default BeeGFS management configuration:
# beegfs_ha_beegfs_mgmtd_conf_resource_group_options:
#  <beegfs-mgmt.conf:key>:<beegfs-mgmt.conf:value>
floating_ips:
  - i1b: 100.127.101.0/16
  - i2b: 100.128.102.0/16
beegfs_service: management
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 1
            owning_controller: A
```

3. 「group_vars/`」の下で、次のテンプレートを使用して「meta_01」から「meta_08」までのリソースグループのファイルを作成し、以下の表を参照する各サービスのプレースホルダ値を入力します。

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET> # Example: i1b:192.168.120.1/16
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 21.25 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

ボリュームサイズは、ストレージプール（ボリュームグループとも呼ばれる）全体に対する割合で指定します。SSDのオーバープロビジョニングのためのスペースを確保するために、各プールにある程度の空き容量を確保することを強く推奨します（詳細については、を参照してください） "NetApp EF600アレイの概要"）。ストレージプール'beegfs_m1_m2_m3_m6"は'管理サービス用のプールの容量の1%も割り当てますしたがって'ストレージ・プール内のメタデータ・ボリュームでは'beegfs_m1_m2_m5_m6'1.92TBまたは3.84TBのドライブを使用している場合'この値を21.25'7.65TBドライブの場合は22.25'15.3TBドライブの場合は'23.75'に設定します

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
meta_01.yml	8015	i1b：100.127.101.1/16 i2b：100.128.102.1/16	0	ictad22a01	beegfs_m1_m2_m5_m6	A
meta_02.yml	8025	i2b：100.128.102.2/16 i1b：100.127.101.2/16	0	ictad22a01	beegfs_m1_m2_m5_m6	B

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
meta_03.yml	8035	i3b：100.127.101.3/16 i4b：100.128.102.3/16	1.	ictad22a02	beegfs_m3_m4_m7_M8	A
meta_04.yml	8045	i4b：100.128.102.4/16 i3b：100.127.101.4/16	1.	ictad22a02	beegfs_m3_m4_m7_M8	B
meta_05.yml	8055	i1b：100.127.101.5/16 i2b：100.128.102.5/16	0	ictad22a01	beegfs_m1_m2_m5_m6	A
meta_06.yml	8065	i2b：100.128.102.6/16 i1b：100.127.101.6/16	0	ictad22a01	beegfs_m1_m2_m5_m6	B
meta_07.yml	8075	i3b：100.127.101.7 / 16 i4b：100.128.102.7 / 16	1.	ictad22a02	beegfs_m3_m4_m7_M8	A
meta_08.yml	8085	i4b：100.128.102.8/16 i3b：100.127.101.8/16	1.	ictad22a02	beegfs_m3_m4_m7_M8	B

4. 「group_vars/`」の下で、以下のテンプレートを使用して「stor_01」から「stor_08」のリソースグループ用のファイルを作成し、例を参照する各サービスのプレースホルダ値を入力します。

```
# stor_0X - BeeGFS HA Storage Resource
Groupbeegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!             owning_controller:
<OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

正しいサイズについては、を参照してください "ストレージプールのオーバープロビジョニングの割合を推奨します"。

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
STOR_01.yml	8013	i1b：100.127.103.1/16 i2b：100.128.104.1/16	0	ictad22a01	beegfs_s1_s2	A
STOR_02.yml	8023	i2b：100.128.104.2/16 i1b：100.127.103.2/16	0	ictad22a01	beegfs_s1_s2	B
STOR_03.yml	8033	i3b：100.127.103.3／16 i4b：100.128.104.3／16	1.	ictad22a02	beegfs_s3_s4	A

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
STOR_04.yml	8043	i4b：100.128.104.4/16 i3b：100.127.103.4/16	1.	ictad22a02	beegfs_s3_s4	B
STOR_05.yml	8053	i1b：100.127.103.5/16 i2b：100.128.104.5/16	0	ictad22a01	beegfs_s5_s6	A
STOR_06.yml	8063	i2b：100.128.104.6/16 i1b：100.127.103.6/16	0	ictad22a01	beegfs_s5_s6	B
STOR_07.yml	8073	i3b：100.127.103.7/16 i4b：100.128.104.7/16	1.	ictad22a02	beegfs_s7_s8	A
STOR_08.yml	8083	i4b：100.128.104.8/16 i3b：100.127.103.8/16	1.	ictad22a02	beegfs_s7_s8	B

手順**3**：メタデータとストレージのビルディングブロックのインベントリを設定する

以下の手順では、BeeGFSメタデータとストレージビルディングブロックにAnsibleインベントリを設定する方法について説明します。

手順

1. 'inventory.yml'で'既存の構成の下に次のパラメータを入力します

```
        meta_09:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_09:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_10:
          hosts:
            ictad22h03:
```

```
            ictad22h04:
        stor_10:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_11:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_11:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_12:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_12:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_13:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_13:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_14:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_14:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_15:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_15:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_16:
```

```
          hosts:
            ictad22h04:
            ictad22h03:
        stor_16:
          hosts:
            ictad22h04:
            ictad22h03:
```

2. 「group_vars/`」の下で、次のテンプレートを使用して「meta_09」から「meta_16」までのリソースグループのファイルを作成し、例を参照する各サービスのプレースホルダ値を入力します。

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.5 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

正しいサイズについては、を参照してください "ストレージプールのオーバープロビジョニングの割合を推奨します"。

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
meta_09.yml	8015	i1b：100.127.101.9 / 16 i2b：100.128.102.9 / 16	0	ictad22a03	beegfs_m9_m10_m13_M14	A

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
meta_10.yml	8025	i2b：100.128.102.10/16 i1b：100.127.101.10/16	0	ictad22a03	beegfs_m9_m10_m13_M14	B
meta_11.yml	8035	i3b：100.127.101.11/16 i4b：100.128.102.11/16	1.	ictad22a04	BEegfs_M11_M12_M15_M16	A
meta_12.yml	8045	i4b：100.128.102.0/16 i3b：100.127.101.2/16	1.	ictad22a04	BEegfs_M11_M12_M15_M16	B
meta_13.yml	8055	i1b：100.127.101.13 / 16 i2b：100.128.102.13 / 16	0	ictad22a03	beegfs_m9_m10_m13_M14	A
meta_14.yml	8065	i2b：100.128.102.14 / 16 i1b：100.127.101.14 / 16	0	ictad22a03	beegfs_m9_m10_m13_M14	B
meta_15.yml	8075	i3b：100.127.101.15/16 i4b：100.128.102.15/16	1.	ictad22a04	BEegfs_M11_M12_M15_M16	A
meta_16.yml	8085	i4b：100.128.102.16/16 i3b：100.127.101.16/16	1.	ictad22a04	BEegfs_M11_M12_M15_M16	B

3. 「group_vars/」の下で、「stor_09」から「stor_16」までのリソースグループ用のファイルを作成し、例を参照する各サービスのプレースホルダ値を入力します。

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

正しいサイズについては、を参照してください "ストレージプールのオーバープロビジョニングの割合を推奨します"。

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
STOR_09.yml	8013	i1b：100.127.103.9/16 i2b：100.128.104.9/16	0	ictad22a03	beegfs_s9_s10	A
STOR_10.yml	8023	i2b：100.128.104.10 / 16 i1b：100.127.103.10 / 16	0	ictad22a03	beegfs_s9_s10	B
STOR_11.yml	8033	i3b：100.127.103.11/16 i4b：100.128.104.11 /16	1.	ictad22a04	beegfs_s11_s12を指定します	A

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
STOR_12.yml	8043	i4b：100.128.104.12/16 i3b：100.127.103.12/16	1.	ictad22a04	beegfs_s11_s12を指定します	B
STOR_13.yml	8053	i1b：100.127.103.13 / 16 i2b：100.128.104.13 / 16	0	ictad22a03	beegfs_S13_s14	A
STOR_14.yml	8063	i2b：100.128.104.14/16 i1b：100.127.103.14/16	0	ictad22a03	beegfs_S13_s14	B
STOR_15.yml	8073	i3b：100.127.103.15/16 i4b：100.128.104.15/16	1.	ictad22a04	beegfs_s15_s16	A
STOR_16.yml	8083	i4b：100.128.104.16/16 i3b：100.127.103.16/16	1.	ictad22a04	beegfs_s15_s16	B

手順**4**：ストレージ専用のビルディングブロックのインベントリを設定する

以下の手順では、BeeGFSストレージ専用ビルディングブロックのAnsibleインベントリを設定する方法について説明します。メタデータとストレージのみのビルディング・ブロックの構成を設定する場合の主な違いは'すべてのメタデータ・リソース・グループを省略し'各ストレージ・プールの基準ドライブ数を10から12に変更することです

手順

1. 'inventory.yml'で'既存の構成の下に次のパラメータを入力します

```
    # ictad22h05/ictad22h06 HA Pair (storage only building block):
      stor_17:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_18:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_19:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_20:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_21:
        hosts:
          ictad22h06:
          ictad22h05:
      stor_22:
        hosts:
          ictad22h06:
          ictad22h05:
      stor_23:
        hosts:
          ictad22h06:
          ictad22h05:
      stor_24:
        hosts:
          ictad22h06:
          ictad22h05:
```

2. 「group_vars/」の下で、以下のテンプレートを使用して「`stor_17`~`stor_24`のリソースグループのファイルを作成し、例を参照する各サービスのプレースホルダ値を入力します。

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 12
        common_volume_configuration:
          segment_size_kb: 512
        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50
            owning_controller: <OWNING CONTROLLER>
```

正しいサイズについては、を参照してください "ストレージプールのオーバープロビジョニングの割合を推奨します"。

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
STOR_17.yml	8013	i1b：100.127.103.17/16 i2b：100.128.104.17/16	0	ictad22a05	beegfs_s17_s18	A
STOR_18.yml	8023	i2b：100.128.104.18/16 i1b：100.127.103.18/16	0	ictad22a05	beegfs_s17_s18	B
STOR_19.yml	8033	i3b：100.127.103.19/16 i4b：100.128.104.19/16	1.	ictad22a06	beegfs_s19_s20	A

ファイル名	ポート	フローティングIP	NUMAゾーン	ブロックノード	ストレージプール	所有コントローラ
STOR_20.yml	8043	i4b：100.128.104.20/16 i3b：100.127.103.20/16	1.	ictad22a06	beegfs_s19_s20	B
STOR_21.yml	8053	i1b：100.127.103.21 / 16 i2b：100.128.104.21 / 16	0	ictad22a05	beegfs_S21_s22	A
STOR_22.yml	8063	i2b：100.128.104.22 / 16 i1b：100.127.103.22 / 16	0	ictad22a05	beegfs_S21_s22	B
STOR_23.yml	8073	i3b：100.127.103.23/16 i4b：100.128.104.23/16	1.	ictad22a06	beegfs_S23_s24	A
STOR_24.yml	8083	i4b：100.128.104.24/16 i3b：100.127.103.24/16	1.	ictad22a06	beegfs_S23_s24	B

BeeGFSを導入します

構成の導入と管理には、Ansibleで実行するタスクが含まれた1つ以上のプレイブックを実行し、システム全体を目的の状態にする必要があります。

すべてのタスクを1つのプレイブックに含めることができますが、複雑なシステムでは、この作業が管理しにくくなります。Ansibleを使用すると、再利用可能なプレイブックと関連コンテンツ（デフォルトの変数、タスク、ハンドラなど）をパッケージ化する方法でロールを作成して配布できます。詳細については、Ansibleのドキュメントを参照してください "ロール"。

多くの場合、ロールは関連するロールとモジュールを含むAnsibleコレクションの一部として配布されます。このため、このプレイブックは、主に、NetApp Eシリーズの各種Ansibleコレクションに分散された複数のロールをインポートするだけです。

> 現在、2ノードクラスタとのクォーラムの確立時に問題が発生しないように、別のクォーラムデバイスをTiebreakerとして設定している場合を除き、BeeGFSを導入するには少なくとも2つのビルディングブロック（4つのファイルノード）が必要です。

手順

1. 新しい'playbook.yml`ファイルを作成し'次のものを含めます

```
# BeeGFS HA (High Availability) cluster playbook.
- hosts: eseries_storage_systems
  gather_facts: false
  collections:
    - netapp_eseries.santricity
  tasks:
    - name: Configure NetApp E-Series block nodes.
      import_role:
        name: nar_santricity_management
- hosts: all
  any_errors_fatal: true
  gather_facts: false
  collections:
    - netapp_eseries.beegfs
  pre_tasks:
    - name: Ensure a supported version of Python is available on all
file nodes.
      block:
        - name: Check if python is installed.
          failed_when: false
          changed_when: false
          raw: python --version
          register: python_version
        - name: Check if python3 is installed.
          raw: python3 --version
          failed_when: false
          changed_when: false
          register: python3_version
          when: 'python_version["rc"] != 0 or (python_version["stdout"]
| regex_replace("Python ", "")) is not version("3.0", ">=")'
        - name: Install python3 if needed.
          raw: |
            id=$(grep "^ID=" /etc/*release* | cut -d= -f 2 | tr -d '"')
            case $id in
              ubuntu) sudo apt install python3 ;;
              rhel|centos) sudo yum -y install python3 ;;
              sles) sudo zypper install python3 ;;
            esac
          args:
            executable: /bin/bash
          register: python3_install
          when: python_version['rc'] != 0 and python3_version['rc'] != 0
          become: true
        - name: Create a symbolic link to python from python3.
          raw: ln -s /usr/bin/python3 /usr/bin/python
```

```
          become: true
          when: python_version['rc'] != 0
      when: inventory_hostname not in
groups[beegfs_ha_ansible_storage_group]
    - name: Verify any provided tags are supported.
      fail:
        msg: "{{ item }} tag is not a supported BeeGFS HA tag. Rerun
your playbook command with --list-tags to see all valid playbook tags."
      when: 'item not in ["all", "storage", "beegfs_ha",
"beegfs_ha_package", "beegfs_ha_configure",
"beegfs_ha_configure_resource", "beegfs_ha_performance_tuning",
"beegfs_ha_backup", "beegfs_ha_client"]'
      loop: "{{ ansible_run_tags }}"
  tasks:
    - name: Verify before proceeding.
      pause:
        prompt: "Are you ready to proceed with running the BeeGFS HA
role? Depending on the size of the deployment and network performance
between the Ansible control node and BeeGFS file and block nodes this
can take awhile (10+ minutes) to complete."
    - name: Verify the BeeGFS HA cluster is properly deployed.
      import_role:
        name: beegfs_ha_7_2
```

このプレイブックは、Python 3がファイルノードにインストールされていることを確認し、提供されたAnsibleタグがサポートされていることを確認するいくつかの「pre_ttasks」を実行します。

2. BeeGFSを導入する準備ができたら'Ansibleプレイブックコマンドを使用してインベントリとプレイブックファイルを作成します

   配備ではすべての「pre_tasks」が実行され、ユーザーの確認を求めるプロンプトが表示された後、実際のBeeGFS配備に進みます。

   次のコマンドを実行して、必要に応じてフォークの数を調整します（以下の注記を参照）。

```
ansible-playbook -i inventory.yml playbook.yml --forks 20
```

特に大規模な展開では、forkパラメータを使用してデフォルトのfork（5）をオーバーライドし、Ansibleがパラレルに構成するホストの数を増やすことをお勧めします。（詳細については、を参照してください "Ansibleのパフォーマンス調整" および "プレイブックの実行を制御する".） 最大値の設定は、Ansibleの制御ノードで使用可能な処理能力によって異なります。上記の例では、CPUを4つ搭載した仮想Ansibleコントロールノード（インテル（R）Xeon（R）Gold 6146 CPU @ 3.20GHz）上で20を実行しています。

導入のサイズと、Ansible制御ノードとBeeGFSファイルおよびブロックノードの間のネットワークパフォ

ーマンスによって、導入時間が異なる場合があります。

BeeGFSクライアントを設定します

コンピューティングノードやGPUノードなど、BeeGFSファイルシステムにアクセスする必要のあるホストにBeeGFSクライアントをインストールして設定する必要があります。このタスクでは、AnsibleとBeeGFSコレクションを使用できます。

手順

1. 必要に応じて、Ansibleコントロールノードから、BeeGFSクライアントとして設定する各ホストにパスワードなしのSSHを設定します。

   「ssh-copy-id」<user>@<hostname_or_ip>`

2. 「host_vars/`」の下で、「<hostname>.yml」という名前のBeeGFSクライアントごとに、次の内容でファイルを作成し、環境に適した情報をプレースホルダテキストに入力します。

```
# BeeGFS Client
ansible_host: <MANAGEMENT_IP>
# OPTIONAL: If you want to use the NetApp E-Series Host Collection's
IPoIB role to configure InfiniBand interfaces for clients to connect to
BeeGFS file systems:
eseries_ipoib_interfaces:
  - name: <INTERFACE>
    address: <IP>/<SUBNET_MASK> # Example: 100.127.1. 1/16
  - name: <INTERFACE>0
    address: <IP>/<SUBNET_MASK>
```

   現在、2つのストレージIPoIBサブネットごとに1つずつ、各クライアントに2つのInfiniBandインターフェイスを設定する必要があります。ここに記載されているBeeGFSサービスごとに、サンプルのサブネットと推奨範囲を使用する場合、クライアントには、「100.127.1」の範囲で設定されたインターフェイスが1つ必要です。0〜100.127.99.255'およびその他の値は'100.128.1です0〜100.128。99.255`.

3. 新しいファイル'client_inventory.yml'を作成し'上部に次のパラメータを設定します

```
# BeeGFS client inventory.
all:
  vars:
    ansible_ssh_user: <USER> # This is the user Ansible should use to
connect to each client.
    ansible_become_password: <PASSWORD> # This is the password Ansible
will use for privilege escalation, and requires the ansible_ssh_user be
root, or have sudo privileges.
The defaults set by the BeeGFS HA role are based on the testing
performed as part of this NetApp Verified Architecture and differ from
the typical BeeGFS client defaults.
```

パスワードをプレーンテキストで保存しないでください。代わりにAnsible Vaultを使用します（のAnsibleのドキュメントを参照してください） "Ansible Vaultを使用したコンテンツの暗号化")または'プレイブックを実行するときに'--Ask -bece-pass`オプションを使用します

4. 「client_inventory.yml」ファイルで、「beegfs_clients」グループの下にBeeGFSクライアントとして設定する必要があるすべてのホストを一覧表示し、BeeGFSクライアントカーネルモジュールの構築に必要な追加の設定を指定します。

```
  children:
    # Ansible group representing all BeeGFS clients:
    beegfs_clients:
      hosts:
        ictad21h01:
        ictad21h02:
        ictad21h03:
        ictad21h04:
        ictad21h05:
        ictad21h06:
        ictad21h07:
        ictad21h08:
        ictad21h09:
        ictad21h10:
      vars:
        # OPTION 1: If you're using the Mellanox OFED drivers and they
are already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        beegfs_client_ofed_enable: True
        beegfs_client_ofed_include_path:
"/usr/src/ofa_kernel/default/include"
        # OPTION 2: If you're using inbox IB/RDMA drivers and they are
already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        # OPTION 3: If you want to use inbox IB/RDMA drivers and need
them installed/configured.
        eseries_ib_skip: False # Default value.
        beegfs_client_ofed_enable: False # Default value.
```

Mellanox OFEDドライバを使用する場合は'beegfs_client_OFED_include_path'使用しているLinuxインストールに適したヘッダーインクルード・パスを指していることを確認してください詳細については、のBeeGFSのマニュアルを参照してください "RDMAのサポート"。

5. 'client_inventory.yml'ファイルに'以前に定義したすべての変数の一番下にマウントするBeeGFSファイル・システムを一覧表示します

```
        beegfs_client_mounts:
          - sysMgmtdHost: 100.127.101.0 # Primary IP of the BeeGFS
management service.
            mount_point: /mnt/beegfs    # Path to mount BeeGFS on the
client.
            connInterfaces:
              - <INTERFACE> # Example: ibs4f1
              - <INTERFACE>
            beegfs_client_config:
              # Maximum number of simultaneous connections to the same
node.
              connMaxInternodeNum: 128 # BeeGFS Client Default: 12
              # Allocates the number of buffers for transferring IO.
              connRDMABufNum: 36 # BeeGFS Client Default: 70
              # Size of each allocated RDMA buffer
              connRDMABufSize: 65536 # BeeGFS Client Default: 8192
              # Required when using the BeeGFS client with the shared-
disk HA solution.
              # This does require BeeGFS targets be mounted in the
default “sync” mode.
              # See the documentation included with the BeeGFS client
role for full details.
              sysSessionChecksEnabled: false
```

「beegfs_client_config」は、テストされた設定を表します。すべてのオプションの包括的な概要については’netapp_eseries.beegfs`コレクションのbeegfs_client’ロールに付属のマニュアルを参照してくださいこれには、複数のBeeGFSファイルシステムのマウントまたは同じBeeGFSファイルシステムの複数回のマウントに関する詳細が含まれます。

6. 新しい’client_playbook.yml`ファイルを作成し’次のパラメータを設定します

```
# BeeGFS client playbook.
- hosts: beegfs_clients
  any_errors_fatal: true
  gather_facts: true
  collections:
    - netapp_eseries.beegfs
    - netapp_eseries.host
  tasks:
    - name: Ensure IPoIB is configured
      import_role:
        name: ipoib
    - name: Verify the BeeGFS clients are configured.
      import_role:
        name: beegfs_client
```

> 必要なIB/RDMAドライバをインストールし、適切なIPoIBインターフェイスにIPを設定している場合は、「NetApp_eseries.host」コレクションと「IPoIB」ロールのインポートを省略します。

7. クライアントをインストールしてビルドし、BeeGFSをマウントするには、次のコマンドを実行します。

```
ansible-playbook -i client_inventory.yml client_playbook.yml
```

8. BeeGFSファイル・システムを本番環境に配置する前に'任意のクライアントにログインし'beegfs -ffsck --checkfs'を実行して'すべてのノードにアクセスできることと'問題が報告されないことを確認することを強くお勧めします

5つのビルディングブロックを超えた拡張性

PacemakerとCorosyncを5つ以上のビルディングブロック（10個のファイルノード）に拡張できるように設定できます。ただし、大規模なクラスタには欠点があるため、最終的にはPacemakerとCorosyncでは最大32個のノードが必要になります。

ネットアップでは、最大10個のノードについてBeeGFS HAクラスタのみをテストしています。この制限を超える個々のクラスタの拡張は推奨もサポートもされません。ただし、BeeGFSファイルシステムは依然として10ノードをはるかに超える規模に拡張する必要があり、ネットアップはNetApp解決策 のBeeGFSでこれを計上しています。

各ファイルシステムのビルディングブロックのサブセットを含む複数のHAクラスタを導入することで、基盤となるHAクラスタリングメカニズムの推奨制限やハード制限とは無関係に、BeeGFSファイルシステム全体を拡張できます。このシナリオでは、次の手順を実行します。

- 追加のHAクラスタを表す新しいAnsibleインベントリを作成し、別の管理サービスの設定は省略します。代わりに'ha_cluster.yml'追加する各クラスタのbeegfs_ha_gmtd_floating_ip'変数を最初のBeeGFS管理サービスのIPに指定します

- 同じファイルシステムにHAクラスタを追加する場合は、次の点を確認してください。
  - BeeGFSノードIDは一意です。
  - 「group_vars」の下の各サービスに対応するファイル名は、すべてのクラスタで一意です。
  - BeeGFSクライアントとサーバのIPアドレスは'すべてのクラスタで一意です
  - 追加のクラスタを導入または更新する前に、BeeGFS管理サービスを含む最初のHAクラスタが実行されています。
- 各HAクラスタのインベントリを、それぞれのディレクトリツリーで個別に維持します。

  複数のクラスタのインベントリファイルを1つのディレクトリツリーに混在させようとすると、BeeGFS HAロールで特定のクラスタに適用される構成を集約する方法で原因 の問題が発生することがあります。

> 新しいビルディングブロックを作成する前に、HAクラスタごとに5つのビルディングブロックを拡張する必要はありません。多くの場合、クラスタあたりの使用するビルディングブロック数が少なくて済むため、管理が容易です。1つは、各ラックのビルディングブロックをHAクラスタとして構成する方法です。

ストレージプールのオーバープロビジョニングの割合を推奨します

第2世代のビルディングブロックでストレージプールあたりの標準の4ボリューム構成に従う場合は、次の表を参照してください。

次の表に、BeeGFSメタデータまたはストレージ・ターゲットごとの'eseries_storage_pool_configuration'でボリューム・サイズとして使用する推奨パーセンテージを示します。

ドライブサイズ	サイズ
1.92TB	18
3.84TB	21.5
7.68TB	22.5インチ
15.3TB	24

> 上記のガイダンスは、管理サービスが含まれるストレージプールには適用されません。この場合、管理データ用にストレージプールの1%を割り当てるために、25%上のサイズを縮小する必要があります。

これらの値の決定方法については、を参照してください "TR-4800『Appendix A：Understanding SSD持久力とオーバープロビジョニング』"。

法的通知

著作権に関する声明、商標、特許などにアクセスできます。

著作権

http://www.netapp.com/us/legal/copyright.aspx

商標

NetApp 、 NetApp のロゴ、および NetApp の商標ページに記載されているマークは、 NetApp, Inc. の商標です。その他の会社名および製品名は、それぞれの所有者の商標である場合があります。

http://www.netapp.com/us/legal/netapptmlist.aspx

特許

ネットアップが所有する特許の最新リストは、次のサイトで入手できます。

https://www.netapp.com/us/media/patents-page.pdf

プライバシーポリシー

https://www.netapp.com/us/legal/privacypolicy/index.aspx

オープンソース

通知ファイルには、ネットアップソフトウェアで使用されるサードパーティの著作権およびライセンスに関する情報が記載されています。

"E シリーズ / EF シリーズ SANtricity OS に関する通知です"

著作権情報

Copyrightゥ2022 NetApp、Inc. All rights reserved.米国で印刷されていますこのドキュメントは著作権によって保護されています。画像媒体、電子媒体、および写真複写、記録媒体などの機械媒体など、いかなる形式および方法による複製も禁止します。 テープ媒体、または電子検索システムへの保管-著作権所有者の書面による事前承諾なし。

ネットアップの著作物から派生したソフトウェアは、次に示す使用許諾条項および免責条項の対象となります。

このソフトウェアは、ネットアップによって「現状のまま」提供されています。ネットアップは明示的、または商品性および特定目的に対する適合性の暗示的保証を含み、かつこれに限定されないいかなる暗示的な保証も行いません。ネットアップは、いかなる場合でも、間接的、偶発的、特別、懲罰的、または または結果的損害（代替品または代替サービスの調達、使用の損失、データ、利益、またはこれらに限定されないものを含みますが、これらに限定されません。） ただし、契約、厳格責任、または本ソフトウェアの使用に起因する不法行為（過失やその他を含む）のいずれであっても、かかる損害の可能性について知らされていた場合でも、責任の理論に基づいて発生します。

ネットアップは、ここに記載されているすべての製品に対する変更を随時、予告なく行う権利を保有します。ネットアップによる明示的な書面による合意がある場合を除き、ここに記載されている製品の使用により生じる責任および義務に対して、ネットアップは責任を負いません。この製品の使用または購入は、ネットアップの特許権、商標権、またはその他の知的所有権に基づくライセンスの供与とはみなされません。

このマニュアルに記載されている製品は、 1 つ以上の米国特許、その他の国の特許、および出願中の特許によ特許、その他の国の特許、および出願中の特許。

権利の制限について：政府による使用、複製、開示は、 DFARS 252.227-7103 （ 1988 年 10 月）および FAR 52-227-19 （ 1987 年 6 月）の Rights in Technical Data and Computer Software （技術データおよびコンピュータソフトウェアに関する諸権利）条項の（ c ）（ 1 ）（ ii ）項、に規定された制限が適用されます。

商標情報

NetApp、NetAppのロゴ、に記載されているマーク http://www.netapp.com/TM は、NetApp、Inc.の商標です。その他の会社名と製品名は、それを所有する各社の商標である場合があります。